REFERENCE

GATITAS TAPABAÑOS NACATLÁN



Última actualización: 2026-11-03

Índice

1. C++ 2
- 1.1. Configuración 2
  - 1.1.1. Compilación 2
  - 1.1.2. Plantilla 2
- 1.2. STL (Standard Template Library) 3
  - 1.2.1. Vector 3
  - 1.2.2. Multiset 3
  - 1.2.3. Map 3
  - 1.2.4. Priority Queue 3
  - 1.2.5. Deque 3
  - 1.2.6. Bitset 3
- 1.3. Funciones para strings 3
- 1.4. Funciones de `<algorithm>` 4
- 1.5. Otras funciones útiles 5
- 1.6. Constantes 5

2. Estructuras de Datos 5
- 2.1. Ordered Set 5
- 2.2. Segment tree 6
- 2.3. Fenwick tree 8
- 2.4. Min Queue (Deque) 9
- 2.5. Min Queue mediante Dos Min Stacks 10
- 2.6. Priority Queue sobre Estructura de Datos (con Rollback) 11
- 2.7. Chtholly Tree (Range Color / Assignment) 12
- 2.8. RMQ Sparse Table $< O(n), O(1) >$ 13
- 2.9. Sparse Table (RMQ) 14
- 2.10. Sparse Table Disjunta 14
- 2.11. Mo's Algorithm (com Ordenação de Hilbert) 15

3. Ordenamiento y Búsqueda 16
- 3.1. Two Pointers 16
- 3.2. Búsqueda binaria en funciones monótonas 17

4. Teoría de gráficas 17
- 4.1. Graph Traversal 17
  - 4.1.1. Floodfill 17
- 4.2. Disjoin Set Union 19
- 4.3. Small-to-Large Merging (Sack / DSU on Tree) 21
- 4.4. Topological Sort 22
- 4.5. Detección de ciclos 24
- 4.6. Detección de Ciclo (Tortoise and Hare) 25
- 4.7. Lowest Common Ancestor (LCA) 26
- 4.8. Centro y Diámetro de un Árbol 27

4.9. Euler Tour (Intervalos de Entrada/Salida) 28
4.10. Functional Graph (Binary Lifting sobre Ciclos) 28
4.11. Dijkstra 30
4.12. Floyd-Warshall 30
4.13. Johnson (All-Pairs Shortest Path) 31
5. Programación Dinámica 32
5.1. LIS (Longest Increasing Subsequence) con Reconstrucción 33
5.2. LIS (Longest Increasing Subsequence) - Tamaño 33
5.3. Divide and Conquer DP (Particionamento en k subarreglos) 34
5.4. Longest Common Subsequence con Reconstrucción (Hirschberg) 35
5.5. Knapsack (Mochila) con Reconstrucción y Memoria Optimizada 36
6. Teoría de números 38
6.1. Cribas 38
6.2. Miller-Rabin (Test de Primalidad Determinístico) 39
6.3. Euclid's Algorithm 40
6.3.1. Extended Euclid's Algorithm: 40
6.4. Euler's Theorem 41
6.5. Solving Equations 41
7. Combinatoria 41
7.1. Binomial Coefficients 41
8. Strings 41
8.1. Trie 41
8.2. Trie UFMG 42
8.3. String hashing 43
8.4. KMP — Knuth-Morris-Pratt 43
9. Manipulación de bits 44
9.1. Máscara de bits 44
9.1.1. Manipulación individual de bits 44
9.1.2. Working with the rightmost 1-bit 44
9.1.3. Creating useful bit masks 44
9.1.4. Common bit manipulation tricks 44
9.1.5. Bit operations as math alternatives 44
9.1.6. GNU C++ compiler built-in functions 45
9.2. Representing Sets 45
10. Matemáticas 45
10.1. Operaciones básicas con matrices 45
10.2. Division Trick (Floor Division Blocks) 46
11. Teoría 47
11.1. Teoría de números 47
11.1.1. Algunas funciones relevantes 47
11.1.2. Aritmética modular 47
11.2. Combinatoria 47
11.2.1. Números de Fibonacci 47
11.2.2. Números de Catalán 47
11.2.3. Identidades binomiales 48
11.2.4. Estrellas y barras 49
11.2.5. Principio de inclusión-exclusión 49
11.2.6. Derangement (subfactorial) 49
11.3. Primeros términos de algunas sucesiones básicas 50
11.4. Geometría 50
11.4.1. Ternas pitagóricas 50
11.4.2. Trigonometría 50
11.4.3. Producto punto 51
11.5. Sumas y series 51
11.6. DP 51
11.7. Geometría 52
11.8. Grafos 52
12. Otros 53
12.1. Conectividad Dinámica Offline (Divide and Conquer + DSU con Rollback) . . 53

1. C++

1.1. Configuración

- **Imprimir con n decimales:** `cout << fixed << setprecision(n);`

1.1.1. Compilación

cpp/compilation.sh

```
co() {
    g++ -std=c++20 -O2 -Wall -Wextra -Wshadow -Wconversion -o $1 $1.cpp
}

run() {
    co $1 && ./$1;
}
```

1.1.2. Plantilla

cpp/template.h

```
#include <bits/stdc++.h>
```

```
2
3  using namespace std;
4
5  #define endl '\n'
6  #define dbg(x) cout << #x << ": " << x << endl
7  #define sz(x) int((x).size())
8  #define all(v) (v).begin(), (v).end()
9  #define rall(v) (v).rbegin(), (v).rend()
10 #define pb push_back
11 #define mp make_pair
12 #define ff first
13 #define ss second
14
15 using ll = long long;
16 const ll INF = 1e18;
17 const ll MOD = 1e9 + 7;
```

1.2. STL (Standard Template Library)

1.2.1. Vector

`v.insert(it, x)` – $O(n)$: Inserta `x` en la posición apuntada por el iterador `it`
`v.erase(it)` – $O(n)$: Remueve el elemento apuntado por el iterador `it`
`v.erase(it1, it2)` – $O(n)$: Remueve los valores en el intervalo `[it1, it2)` de iteradores

1.2.2. Multiset

En un multiset, `erase` remueve *todas* las copias de un valor, y `count` devuelve el número de copias de un valor.
Remover una solo ocurrencia de un valor x: `s.erase(s.find(x))`

Las funciones `count` y `erase` tienen un factor adicional de $O(k)$ donde k es el número de elementos contados/eliminados. En particular, *no* es eficiente contar el número de copias de un valor en un multiset usando la función `count`.

1.2.3. Map

`m.erase(key)` – $O(\log n)$: Remueve la clave `key` del mapa

1.2.4. Priority Queue

En la cola de prioridad por defecto el mayor elemento es el primero en salir.

`pq.push(x)` – $O(\log n)$: Agrega `x` a la cola de prioridad
`pq.pop()` – $O(\log n)$: Remueve el mayor elemento de la cola de prioridad
`pq.top()` – $O(1)$: Devuelve el mayor elemento de la cola de prioridad
`priority_queue<int, vector<int>, greater<int>> pq`: Cola de prioridad en la que el elemento más pequeño es el primero en salir

Si solo necesitamos encontrar o eliminar eficientemente el elemento mínimo o máximo, es buena idea usar una cola de prioridad en lugar de un set o multiset.

1.2.5. Deque

Es un arreglo dinámico que puede manipular eficientemente ambos extremos de la estructura.
`d.push_back(x)` – $O(1)$: Agrega `x` al final
`d.push_front(x)` – $O(1)$: Agrega `x` al inicio
`d.pop_back()` – $O(1)$: Remueve el elemento del final
`d.pop_front()` – $O(1)$: Remueve el elemento del inicio

1.2.6. Bitset

Sirve para representar máscaras de bits cuando un entero no es suficiente. Posee operaciones optimizadas por el procesador.
`bitset<N> a` – $O(1)$: Crea un bitset de tamaño `N` (`N` debe ser constante)
`a[i]` – $O(1)$: Retorna el valor del bit en la posición `i`
`a.count()` – $O(N/w)$: Retorna el número de bits con 1
`a.size()` – $O(1)$: Retorna el tamaño del bitset
`a.set(i)` – $O(1)$: Pone el bit en la posición `i` a 1
`a.set()` – $O(N/w)$: Pone todos los bits a 1
`a.reset(i)` – $O(1)$: Pone el bit en la posición `i` a 0
`a.reset()` – $O(N/w)$: Pone todos los bits a 0
`a.flip(i)` – $O(1)$: Invierte el bit en la posición `i`
`a.flip()` – $O(N/w)$: Invierte todos los bits
`a.to_ullong()` – $O(N/w)$: Retorna el valor del bitset como un entero sin signo (si el tamaño del bitset es mayor al número de bits de un entero, solo se consideran los bits menos significativos)
Imprimir un entero como binario: `cout << bitset<8>(mask) << "\n";`

El bitset también soporta operaciones como: `&`, `|`, `^`, `~`, `<<`, `>>`, entre otras

Obs: w es el tamaño de palabra del procesador, en general 32 o 64 bits.

1.3. Funciones para strings

`s.substr(i, len)` – $O(n)$: Retorna el substring que inicia en `i` y tiene longitud `len`. Si `len` se omite va hasta el final.
`s.find(str, pos)`, `s.rfind(str, pos)` – $O(nm)$: Busca la primera (`find`) o última (`rfind`) ocurrencia de `str` en `s` a partir de `pos`. Retorna `string::npos` si no existe.

```
s = "abcabcabc";
s.find("bc");  // 1
s.rfind("bc"); // 7
```

`s.replace(pos, len, str)` – $O(n)$: Reemplaza **`len`** caracteres desde **`pos`** con la cadena **`str`**.

```
s = "hola mundo";
s.replace(5, 5, "C++");    // "hola C++"
s.replace(0, 4, "adios");  // "adios C++"
```

`s.erase(pos, len)`, **`s.erase(iterator)`** – $O(n)$: Elimina **`len`** caracteres desde **`pos`**, o el carácter apuntado por un **`iterator`**. Muy útil para remover caracteres/subcadenas no deseados durante el procesamiento.

```
s = "hola mundo";
s.erase(4, 6);   // "hola"
```

`s.insert(pos, str)`, **`s.insert(pos, n, ch)`** – $O(n)$: Inserta **`str`** en la posición **`pos`**, desplazando el resto. También puede insertar **`n`** copias de un carácter **`ch`**.

```
s = "holamundo";
s.insert(4, " ");    // "hola mundo"

t = "abc";
t.insert(1, 3, '*'); // "a***bc"
```

`s.find_first_of(chars, pos)`, **`s.find_last_of(chars, pos)`** – $O(nm)$: Encuentra la posición del primer o último carácter de **`s`** que pertenezca al conjunto **`chars`**. Ideal para parsing de múltiples delimitadores en una sola llamada.

```
s = "hola,mundo;cpp";
s.find_first_of(";,"); // 4 (',')
s.find_last_of(";,");  // 9 (';')
```

`s.compare(str)`, **`s.compare(pos, len, str)`** – $O(n)$: Compara lexicográficamente. Retorna **`0`** si son iguales, **`< 0`** si **`s`** < **`str`** y **`> 0`** si **`s`** > **`str`**. Permite comparar subcadenas sin extraerlas, evitando allocaciones innecesarias.

```
"abc".compare("abc") == 0;  // igual
"abc".compare("abd") < 0;   // menor

s = "hola mundo";
s.compare(5, 5, "mundo") == 0;  // true
```

`count(all(s), ch)` – $O(n)$: Cuenta las ocurrencias de un carácter **`ch`** en el **`string`**.

```
s = "banana";
count(all(s), 'a');  // 3
count(all(s), 'n');  // 2
count(all(s), 'b');  // 1
```

`isalpha(c)`, **`isdigit(c)`**, **`isalnum(c)`**, **`isspace(c)`** – $O(1)$: Verifican si un carácter **`c`** es letra, dígito, alfanumérico o espacio respectivamente.

`s.erase(unique(all(s)), s.end())` – $O(n)$: Elimina caracteres duplicados consecutivos. Si se aplica sobre un string ordenado, elimina todos los duplicados.

```
s = "aabbccddee";
s.erase(unique(all(s)), s.end());
// "abcde"

// Todos los duplicados (con sort previo)
t = "bcaabbcc";
sort(all(t));   // "aabbbbcc"
t.erase(unique(all(t)), t.end()); // "abc"
```

1.4. Funciones de `<algorithm>`

`fill(all(v), x)` – $O(n)$: Rellena **`v`** con **`x`**

`binary_search(begin, end, val)` – $O(\log n)$: Retorna **`true`** si **`val`** esta en el rango ordenado.

```
if (binary_search(all(v), 5))
cout << "Existe";
```

`next_permutation(begin, end)` – $O(n)$: Genera la siguiente permutación lexicográfica. Caso especial: retorna **`false`** y deja el rango en la primera permutación (la ordenada) si ya era la última.

```
vector<int> v = {3,2,1};
bool hay = next_permutation(all(v));
// hay == false, v = {1,2,3}
```

`max_element(begin, end)` – $O(n)$: Retorna iterador al maximo. Para varios maximos, retorna el primero.

```
auto it = max_element(all(v));
if (it != v.end())
    cout << *it;
```

`min_element(begin, end)` – $O(n)$: Retorna iterador al minimo. Para varios mínimos, retorna el primero.

```
int min_val = *min_element(all(v));
// asumiendo no vacio
```

`minmax_element(begin, end)` – $O(n)$: Retorna **`pair<minIt, maxIt>`**. Caso especial: con elementos iguales, **`min`** y **`max`** pueden apuntar al mismo elemento. Rango vacio -> ambos **`end()`**.

```
auto [mn, mx] = minmax_element(all(v));
```

`count(begin, end, val)` – $O(n)$: Cuenta ocurrencias de **`val`**.

```
int cnt = count(all(v), 2);
// puede ser 0
```

`count_if(begin, end, pred)` – $O(n)$: Cuenta elementos que cumplen **`pred`**.

```
int pares = count_if(all(v), [](int x) {
    return x % 2 == 0;
});
```

`find(begin, end, val)` – $O(n)$: Retorna un iterador para la primera ocurrencia del elemento `x` en `v`, o `v.end()` si no existe

```
auto it = find(all(v), 3);
if (it != v.end())
    cout << "Encontrado en indice " << it - v.begin();
```

`rotate(begin, new_begin, end)` – $O(n)$: Rota de forma que `new_begin` pasa a ser el primer elemento. Caso especial: Si `new_begin == begin` o `new_begin == end`, no hay rotación efectiva.

`unique(begin, end)` – $O(n)$: "Compacta" elementos duplicados consecutivos moviendo las primeras ocurrencias al frente. Caso especial: No cambia el tamaño del contenedor. Devuelve iterador al nuevo final lógico; se debe usar `erase` para eliminar los sobrantes.

```
vector<int> v = {1,1,2,2,2,3};
v.erase(unique(all(v)), v.end());
// {1,2,3}
```

`merge(b1, e1, b2, e2, out)` – $O(n+m)$: Fusiona dos rangos ordenados en uno solo. Caso especial: Si los rangos de entrada no están ordenados, el comportamiento es indefinido.

```
vector<int> a = {1,3,5}, b = {2,4,6};
vector<int> c(6);
merge(all(a), all(b), c.begin());
// {1,2,3,4,5,6}
```

`partition(begin, end, pred)` – $O(n)$: Reordena: primero los que cumplen `pred`, luego el resto. Caso especial: No garantiza el orden relativo dentro de cada grupo. Usar `stable_partition` si se necesita estabilidad.

```
auto it = partition(all(v), [](int x) {
    return x % 2 == 0;
});
// pares al frente, impares al final
```

1.5. Otras funciones útiles

`round(x)` – $O(1)$: Retorna el entero más próximo a `x`

1.6. Constantes

Constante	Nombre	Valor aproximado
π	`M_PI`	3.141592...
$\pi/2$	`M_PI_2`	1.570796...
$\pi/4$	`M_PI_4`	0.785398...
$1/\pi$	`M_1_PI`	0.318309...
$2/\pi$	`M_2_PI`	0.636619...
$2/\sqrt{\pi}$	`M_2_SQRTPI`	1.128379...
$\sqrt{2}$	`M_SQRT2`	1.414213...
$1/\sqrt{2}$	`M_SQRT1_2`	0.707106...
e	`M_E`	2.718281...
$\log_2 e$	`M_LOG2E`	1.442695...
$\log_{10} e$	`M_LOG10E`	0.434294...
$\ln 2$	`M_LN2`	0.693147...
$\ln 10$	`M_LN10`	2.302585...

2. Estructuras de Datos

2.1. Ordered Set

TIME	SPACE
$O(\log n)$	$O(n)$

Estructura de datos basada en un árbol rojo-negro (**Policy-Based Data Structures**) que extiende las funcionalidades de un `std::set` convencional de C++. Además de soportar las operaciones estándar de un conjunto, permite consultar estadísticas de orden en tiempo logarítmico.

Funcionalidades adicionales incluidas:

- `find_by_order(k)`: Retorna un iterador al k-ésimo elemento más pequeño (indexado en cero, donde `k = 0` es el elemento mínimo).
- `order_of_key(k)`: Retorna la cantidad de elementos estrictamente menores que `k`.

Soporte para Multiset: Para simular un multiset, se debe usar `ord_set<pair<int, int>>` utilizando el segundo parámetro como un identificador único para diferenciar elementos con valores iguales. La consulta `s.order_of_key({k, -INF})` devolverá la cantidad total de elementos estrictamente menores que `k`.

estructuras/ordered_set.cpp

```
1  #include <ext/pb_ds/assoc_container.hpp>
2  #include <ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4  template <class T>
5      using ordered_set = tree<
6          T,
```

```
        null_type,
        less<T>,
        rb_tree_tag,
        tree_order_statistics_node_update
    >;

// para declarar:
// ord_set<int> s;
// coisas do set normal funcionam:
// for (auto i : s) cout << i << endl;
// cout << s.size() << endl;
// k-esimo maior elemento O(log|s|):
// k=0: menor elemento
// cout << *s.find_by_order(k) << endl;
// quantos sao menores do que k O(log|s|):
// cout << s.order_of_key(k) << endl;

// Para fazer um multiset, tem que
// usar ord_set<pair<int, int>> com o
// segundo parametro sendo algo para diferenciar
// os ementos iguais.
// s.order_of_key({k, -INF}) vai retornar o
// numero de elementos < k
```

2.2. Segment tree

TIME
- $O(n)$ for building
- $O(\log n)$ for queries and updates

estructuras/segment-tree.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const int MAXN = 2e5;
ll a[MAXN];
struct segtree {
    int n;
    vector<ll> tree;

    segtree(int n) : n(n) {
        tree.resize(4 * n);
        build(1, 0, n - 1);
    }

    void build(int v, int lx, int rx) {
        if (lx == rx) {
            tree[v] = a[lx];
            return;
        }

        int m = (lx + rx) / 2;
        build(2 * v, lx, m);
        build(2 * v + 1, m + 1, rx);
        tree[v] = tree[2 * v] + tree[2 * v + 1];
    }

    void update(int idx, int new_val, int v, int lx, int rx) {
        if (lx == rx) {
            tree[v] = a[idx] = new_val;
            return;
        }

        int m = (lx + rx) / 2;
        if (idx <= m) update(idx, new_val, 2 * v, lx, m);
        else          update(idx, new_val, 2 * v + 1, m + 1, rx);
        tree[v] = tree[2 * v] + tree[2 * v + 1];
    }

    ll query(int l, int r, int v, int lx, int rx) {
        if (l > rx or r < lx or l > r) return 0;
        if (l <= lx and rx <= r) return tree[v];

        int m = (lx + rx) / 2;
        return    query(l, r, 2 * v, lx, m)
                + query(l, r, 2 * v + 1, m + 1, rx);
    }

    void update(int pos, int new_val) {
        update(pos, new_val, 1, 0, n - 1);
```

```
    }

    ll query(int l, int r) {
        return query(l, r, 1, 0, n - 1);
    }
};
```

Segment Tree con Lazy Propagation (Suma en Rango)

TIME	SPACE
build : $O(n)$ query : $O(\log n)$ update : $O(\log n)$	$O(n)$

Segment Tree recursiva con **lazy propagation** para rangos. `query(a, b)` calcula la suma del rango $[a, b]$, y `update(a, b, x)` suma x a cada elemento del rango $[a, b]$. Puede indexarse cada nodo mediante la función $f(l, r) = (l + r) \mid (l \neq r)$, lo que permite usar únicamente $2n$ de memoria en lugar de $4n$.

Funciones principales:

- `build(n2, v2)`: construye el árbol en $O(n)$ a partir de un arreglo `v2` de tamaño `n2`, inicializando cada hoja con su valor correspondiente y propagando las sumas hacia los nodos internos.
- `prop(p, l, r)`: aplica la propagación perezosa (**lazy**) pendiente en el nodo `p`, actualizando su valor acumulado y empujando el `lazy` hacia sus hijos antes de cualquier consulta o actualización.
- `query(a, b)`: retorna la suma de los elementos en el rango $[a, b]$, propagando lazy en el camino recorrido.
- `update(a, b, x)`: incrementa en `x` todos los elementos del rango $[a, b]$, actualizando `lazy` en los nodos totalmente cubiertos.

Extensiones adicionales (para una Segment Tree de **máximo**, no de suma):

- `get_left(a, b, val)`: encuentra en $O(\log n)$ la primera posición dentro de $[a, b]$ cuyo valor sea $\geq$ `val`, o -1 si no existe. Útil para búsquedas de umbral por la izquierda.
- `get_right(a, b, val)`: análogo a `get_left`, pero retorna la última posición dentro de $[a, b]$ cuyo valor sea $\geq$ `val`, o -1 si no existe.

Extensión adicional (para una Segment Tree de **suma** sobre un arreglo `v` no negativo):

- `lower_bound(i, val)`: dado un valor acumulado `val`, encuentra en $O(\log n)$ el mayor índice j tal que $v[i] + v[i + 1] + \ldots + v[j - 1] <$ val. Útil para problemas tipo "cuántos elementos consecutivos caben antes de superar un umbral", análogo a un **binary search** sobre prefijos de suma pero en $O(\log n)$ en vez de $O(\log^2 n)$.

estructuras/segtree_lazy.cpp

```
// SegTree
//
// Recursiva com Lazy Propagation
// Query: soma do range [a, b]
// Update: soma x em cada elemento do range [a, b]
// Pode usar a seguinte funcao para indexar os nohs:
// f(l, r) = (l+r)|(l!=r), usando 2N de memoria
//
// Complexidades:
// build - O(n)
// query - O(log(n))
// update - O(log(n))

namespace seg {
    ll seg[4*MAX], lazy[4*MAX];
    int n, *v;

    ll build(int p=1, int l=0, int r=n-1) {
        lazy[p] = 0;
        if (l == r) return seg[p] = v[l];
        int m = (l+r)/2;
        return seg[p] = build(2*p, l, m) + build(2*p+1, m+1, r);
    }
    void build(int n2, int* v2) {
        n = n2, v = v2;
        build();
    }
    void prop(int p, int l, int r) {
        seg[p] += lazy[p]*(r-l+1);
        if (l != r) lazy[2*p] += lazy[p], lazy[2*p+1] += lazy[p];
        lazy[p] = 0;
    }
    ll query(int a, int b, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) return seg[p];
        if (b < l or r < a) return 0;
        int m = (l+r)/2;
        return query(a, b, 2*p, l, m) + query(a, b, 2*p+1, m+1, r);
    }
```

```
    ll update(int a, int b, int x, int p=1, int l=0, int r=n-1) {
        prop(p, l, r);
        if (a <= l and r <= b) {
            lazy[p] += x;
            prop(p, l, r);
            return seg[p];
        }
        if (b < l or r < a) return seg[p];
        int m = (l+r)/2;
        return seg[p] = update(a, b, x, 2*p, l, m) +
            update(a, b, x, 2*p+1, m+1, r);
    }
};


// Se tiver uma seg de max, da pra descobrir em O(log(n))
// o primeiro e ultimo elemento >= val numa range:

// primeira posicao >= val em [a, b] (ou -1 se nao tem)
int get_left(int a, int b, int val, int p=1, int l=0, int r=n-1) {
    prop(p, l, r);
    if (b < l or r < a or seg[p] < val) return -1;
    if (r == l) return l;
    int m = (l+r)/2;
    int x = get_left(a, b, val, 2*p, l, m);
    if (x != -1) return x;
    return get_left(a, b, val, 2*p+1, m+1, r);
}

// ultima posicao >= val em [a, b] (ou -1 se nao tem)
int get_right(int a, int b, int val, int p=1, int l=0, int r=n-1) {
    prop(p, l, r);
    if (b < l or r < a or seg[p] < val) return -1;
    if (r == l) return l;
    int m = (l+r)/2;
    int x = get_right(a, b, val, 2*p+1, m+1, r);
    if (x != -1) return x;
    return get_right(a, b, val, 2*p, l, m);
}

// Se tiver uma seg de soma sobre um array nao negativo v, da pra
```

```
// descobrir em O(log(n)) o maior j tal que v[i]+v[i+1]+...+v[j-1] < val
int lower_bound(int i, ll& val, int p, int l, int r) {
    prop(p, l, r);
    if (r < i) return n;
    if (i <= l and seg[p] < val) {
        val -= seg[p];
        return n;
    }
    if (l == r) return l;
    int m = (l+r)/2;
    int x = lower_bound(i, val, 2*p, l, m);
    if (x != n) return x;
    return lower_bound(i, val, 2*p+1, m+1, r);
}
```

2.3. Fenwick tree

Binary Indexed Tree (BIT / Fenwick Tree)

TIME	SPACE
$O(\log n)$	$O(n)$

Estructura de datos Binary Indexed Tree (BIT) de suma indexada en cero (0-based). Permite realizar actualizaciones puntuales y consultas de suma por rangos de manera eficiente.

Funciones e interfaces incluidas:

- `Bit(v)`: Construcción lineal en $O(n)$ a partir de un `vector<int>`.
- `update(i, x)`: Suma `x` a la posición `i`.
- `pref(i)`: Retorna la suma acumulada en el intervalo $[0, i]$.
- `query(l, r)`: Retorna la suma en el intervalo $[l, r]$.
- `upper_bound(x)`: Retorna el menor índice `p` tal que `pref(p) > x` ejecutando una búsqueda binaria sobre el árbol en $O(\log n)$.

estructuras/bit.cpp

```
struct Bit {
    int n;
    vector<ll> bit;
    Bit(int _n=0) : n(_n), bit(n + 1) {}
    Bit(vector<int>& v) : n(v.size()), bit(n + 1) {
        for (int i = 1; i <= n; i++) {
            bit[i] += v[i - 1];
```

```
            int j = i + (i & -i);
            if (j <= n) bit[j] += bit[i];
        }
    }
    void update(int i, ll x) { // soma x na posicao i
        for (i++; i <= n; i += i & -i) bit[i] += x;
    }
    ll pref(int i) { // soma [0, i]
        ll ret = 0;
        for (i++; i; i -= i & -i) ret += bit[i];
        return ret;
    }
    ll query(int l, int r) {  // soma [l, r]
        return pref(r) - pref(l - 1);
    }
    int upper_bound(ll x) {
        int p = 0;
        for (int i = __lg(n); i+1; i--)
            if (p + (1<<i) <= n and bit[p + (1<<i)] <= x)
                x -= bit[p += (1 << i)];
        return p;
    }
};
```

Binary Indexed Tree con Actualización por Rangos (Range BIT)

TIME	SPACE
$O(\log n)$	$O(n)$

Variación de la estructura de datos Binary Indexed Tree (BIT) que permite aplicar actualizaciones por rangos y consultas de suma por rangos de manera eficiente mediante el uso de dos arreglos auxiliares (0-based).

Funciones e interfaces incluidas:

- `build(n2, v)`: Construcción en tiempo lineal $O(n)$ a partir de un arreglo estático de enteros.
- `update(l, r, x)`: Suma el valor `x` a todos los elementos en el intervalo $[l, r]$.
- `query(l, r)`: Retorna la suma total de los elementos en el intervalo $[l, r]$.

estructuras/range_bit.cpp

```
// BIT con actualización de rango y suma de rango. 0-indexado.
// update(l,r,x): a[l..r] += x. query(l,r): suma de a[l..r]. O(log n).
// Úsese cuando: "suma x a un rango" y "suma de un rango" intercalados.
```

```
struct RBit {
    int n; vector<ll> b1, b2;
    RBit(int n = 0) : n(n), b1(n + 2), b2(n + 2) {}
    void add(vector<ll>& b, int i, ll x) {
        for (i++; i <= n; i += i & -i) b[i] += x;
    }
    ll get(vector<ll>& b, int i) {
        ll r = 0;
        for (i++; i > 0; i -= i & -i) r += b[i];
        return r;
    }
    void update(int l, int r, ll x) {
        add(b1, l, x);     add(b1, r + 1, -x);
        add(b2, l, x * l); add(b2, r + 1, -x * (r + 1));
    }
    ll pref(int i) { return get(b1, i) * (i + 1) - get(b2, i); }
    ll query(int l, int r) { return pref(r) - pref(l - 1); }
};
// Ej.: RBit b(3); b.update(0,2,5); b.update(1,1,5);
//      b.query(1,1) == 10; b.query(0,2) == 20
```

2.4. Min Queue (Deque)

TIME	SPACE
$O(1)$ amortizado	$O(n)$

Cola (**queue**) que soporta obtener el mínimo actual en $O(1)$, implementada mediante un `deque` de pares `{valor, contador}`. Todas las operaciones son $O(1)$ amortizado.

Funciones principales:

- `push(x)`: inserta `x` al frente de la cola. Antes de insertarlo, elimina del frente del `deque` todos los elementos mayores a `x` (ya que nunca podrán ser el mínimo mientras `x` siga en la cola), acumulando su cantidad en un contador `ct` para no perder el conteo real de elementos al hacer `pop`.
- `pop()`: elimina el elemento más antiguo de la cola. Si el par en la parte trasera del `deque` representa más de un elemento (`second > 1`), simplemente decrementa su contador; si representa solo uno, lo elimina por completo.
- `min()`: retorna el mínimo actual de la cola en $O(1)$, correspondiente al valor almacenado en la parte trasera del `deque`.

Es la base para técnicas de **sliding window minimum** y para estructuras tipo **stack** que soportan mínimo, combinadas para simular una cola.

estructuras/minqueue.cpp

```cpp
// Min queue - deque
//
// Tudo O(1) amortizado

template<class T> struct minqueue {
    deque<pair<T, int>> q;

    void push(T x) {
        int ct = 1;
        while (q.size() and x < q.front().first)
            ct += q.front().second, q.pop_front();
        q.emplace_front(x, ct);
    }
    void pop() {
        if (q.back().second > 1) q.back().second--;
        else q.pop_back();
    }
    T min() { return q.back().first; }
};
```

2.5. Min Queue mediante Dos Min Stacks

TIME	SPACE
$O(1)$ amortizado	$O(n)$

Implementación de una cola con mínimo en $O(1)$ amortizado, construida a partir de dos **stacks** con mínimo (`minstack`). Todas las operaciones son $O(1)$ amortizado.

Min Stack (`minstack`):

- `push(x)`: inserta `x` en la pila, guardando junto a él el mínimo acumulado hasta ese punto (`min(top().second, x)`).
- `top()`: retorna el elemento en el tope de la pila.
- `pop()`: elimina y retorna el elemento en el tope.
- `min()`: retorna el mínimo de toda la pila en $O(1)$, almacenado junto al elemento del tope.

Min Queue (`minqueue`), construida con dos `minstack` (`s1` para inserciones, `s2` para remociones):

- `push(x)`: inserta `x` en `s1`.
- `move()`: si `s2` está vacía, transfiere todos los elementos de `s1` a `s2` (invirtiendo su orden), preparando así el frente de la cola. Es la operación que amortiza el costo, ya que cada elemento se mueve a lo sumo una vez de `s1` a `s2`.
- `front()`: llama a `move()` y retorna el elemento al frente de la cola (`s2.top()`).
- `pop()`: llama a `move()` y elimina el elemento al frente de la cola.
- `min()`: retorna el mínimo global comparando el mínimo de `s1` y el mínimo de `s2` (considerando el caso en que alguna esté vacía).

Es una alternativa a la implementación con `deque` (ver **Min Queue (Deque)**), útil cuando se prefiere evitar el manejo de contadores de repetición y trabajar directamente con dos pilas.

estructuras/minqueue_stack.cpp

```cpp
// Min queue - stack
//
// Tudo O(1) amortizado

template<class T> struct minstack {
    stack<pair<T, T>> s;

    void push(T x) {
        if (!s.size()) s.push({x, x});
        else s.emplace(x, std::min(s.top().second, x));
    }
    T top() { return s.top().first; }
    T pop() {
        T ans = s.top().first;
        s.pop();
        return ans;
    }
    int size() { return s.size(); }
    T min() { return s.top().second; }
};

template<class T> struct minqueue {
    minstack<T> s1, s2;

    void push(T x) { s1.push(x); }
    void move() {
        if (s2.size()) return;
```

```
        while (s1.size()) {
            T x = s1.pop();
            s2.push(x);
        }
    }
    T front() { return move(), s2.top(); }
    T pop() { return move(), s2.pop(); }
    int size() { return s1.size()+s2.size(); }
    T min() {
        if (!s1.size()) return s2.min();
        else if (!s2.size()) return s1.min();
        return std::min(s1.min(), s2.min());
    }
};
```

2.6. Priority Queue sobre Estructura de Datos (con Rollback)

TIME	SPACE
update : $O(\log(n) + T(n))$ query : $O(T(n))$ pop : $O(\log(n) \cdot T(n))$ amortizado	$O(n + q)$

Estructura **wrapper** que mantiene un conjunto de actualizaciones (`update`) aplicadas sobre otra estructura de datos base `DS`, la cual debe soportar `rollback` y no ser amortizada. Cada actualización se asocia a una prioridad `p`, y en cualquier momento es posible remover la actualización de mayor prioridad mediante `pop()`. Es fundamental que las actualizaciones sean **conmutativas**, es decir, el estado final de la estructura debe ser el mismo sin importar el orden en que se apliquen. Se asume que todas las prioridades son distintas.

Funciones principales:

- `update(u, p)`: aplica la actualización `u` sobre `D` con prioridad `p`, y la registra tanto en `upd` (historial de tuplas `{u, p, idx_in_pos}`) como en `st` (conjunto ordenado de prioridades) para su posterior localización en `pop`.
- `query(a)`: delega la consulta a la estructura base `D.find(a)`, devolviendo el resultado en la complejidad propia de `D` ($T(n)$).
- `pop()`: elimina la actualización con mayor prioridad. Para lograrlo sin romper la conmutatividad, deshace (`rollback`) un bloque de actualizaciones recientes, descarta la de mayor prioridad, y reaplica en orden las restantes (separadas en `small` y `big` según su prioridad relativa). El costo amortizado surge de que el tamaño del bloque a deshacer crece geométricamente, similar a la técnica de **doubling** usada en estructuras persistentes con rollback amortizado.

n es el tamaño de la estructura base `DS` y q el número de actualizaciones almacenadas en el historial (`upd`); $T(n)$ denota la complejidad de `update` sobre `DS`.

estructuras/priority_queue_ds.cpp

```
// Priority Queue DS
//
// Mantem updates aplicados em uma estrutura de dados
// que permita rollback e nao seja amortizada.
// Cada update possui uma prioridade,
// sendo possivel remover o update com maior prioridade.
// Os updates devem ser comutativos, ou seja, o estado
// da estrutura deve ser o mesmo independente da ordem
// que eles sejam aplicados.
//
// Complexidades:
// update - O(log(n) + T(n))
// query - T(n)
// pop - O(log(n) * T(n)) amortizado
//
// onde T(n) eh a complexidade do update

// assumes all priorities are distinct
template<typename DS, typename UPD> struct priority_queue_ds {
    DS D;
    vector<tuple<UPD, int, int>> upd; // {u, p, idx_in_pos}
    set<pair<int, int>> st;
    vector<int> pos;

    priority_queue_ds(int n) : D(n) {}

    void update(UPD u, int p) {
        D.update(u);
        st.emplace(p, pos.size());
        upd.emplace_back(u, p, pos.size());
        pos.push_back(upd.size() - 1);
    }

    int query(int a) {
```

```
        return D.find(a);
    }

    void pop() {
        int k = 1, min_p; // k = number of pops we will do
        vector<tuple<UPD, int, int>> small, big;
        auto it = st.end();
        for (int qt = 0; qt++ < (k+1)/2;) {
            it--;
            min_p = it->first;
            int i = pos[it->second];
            if (qt > 1) big.push_back(upd[i]);
            k = max<int>(k, upd.size() - i);
        }

        for (int i = 0; i < k; i++) {
            D.rollback();
            auto [u, p, idx] = upd.rbegin()[i];
            if (p < min_p) small.emplace_back(u, p, idx);
        }

        st.erase(prev(st.end()));
        upd.erase(upd.end() - k, upd.end());

        small.insert(small.end(), big.rbegin(), big.rend());
        for (auto [u, p, idx] : small) {
            D.update(u);
            upd.emplace_back(u, p, idx);
            pos[idx] = upd.size() - 1;
        }
    }
};
```

2.7. Chtholly Tree (Range Color / Assignment)

TIME	SPACE
update : $O(\log q)$ amortizado query : $O(\log q)$	$O(q)$

Estructura conocida como **Chtholly Tree** (o **Old Driver Tree**), basada en un `set` de intervalos `{l, r, color}` que representa asignaciones de color a rangos. Es eficiente cuando las operaciones de asignación tienden a "fusionar" muchos elementos en un mismo color, ya que el costo amortizado depende de q (número de operaciones), no de n (tamaño del arreglo).

Funciones principales:

- `update(l, r, val)`: colorea el rango $[l, r]$ con el color `val`. Primero divide (**split**) los intervalos existentes que se solapan parcialmente con $[l, r]$ en sus extremos, luego elimina todos los intervalos completamente contenidos en $[l, r]$ (recolectándolos en el vector de retorno `ret`), e inserta el nuevo intervalo $[l, r]$ con color `val`. Retorna la lista de tuplas `{l, r, color}` que fueron sobrescritas por esta operación, lo cual es útil para procesar de forma incremental el efecto del cambio de color sobre otras estructuras (por ejemplo, sumar o restar contribuciones antes de recolorear).
- `query(i)`: retorna el color de la posición `i`, buscando el intervalo del `set` que la contiene; retorna -1 si la posición no ha sido coloreada.

Es especialmente útil en problemas donde las asignaciones de rango "asesinan" (sobrescriben) intervalos anteriores, permitiendo procesar cada elemento del arreglo original a lo sumo un número amortizado de veces a lo largo de todas las operaciones.

estructuras/range_color.cpp

```
// Range color
//
// update(l, r, c) colore o range [l, r] com a cor c,
// e retorna os ranges que foram coloridos {l, r, cor}
// query(i) returna a cor da posicao i
//
// Complexidades (para q operacoes):
// update - O(log(q)) amortizado
// query - O(log(q))

template<typename T> struct color {
    set<tuple<int, int, T>> se;

    vector<tuple<int, int, T>> update(int l, int r, T val) {
        auto it = se.upper_bound({r, INF, val});
        if (it != se.begin() and get<1>(*prev(it)) > r) {
            auto [L, R, V] = *--it;
            se.erase(it);
            se.emplace(L, r, V), se.emplace(r+1, R, V);
        }
```

```
        it = se.lower_bound({l, -INF, val});
        if (it != se.begin() and get<1>(*prev(it)) >= l) {
            auto [L, R, V] = *--it;
            se.erase(it);
            se.emplace(L, l-1, V), it = se.emplace(l, R, V).first;
        }
        vector<tuple<int, int, T>> ret;
        for (; it != se.end() and get<0>(*it) <= r; it = se.erase(it))
            ret.push_back(*it);
        se.emplace(l, r, val);
        return ret;
    }
    T query(int i) {
        auto it = se.upper_bound({i, INF, T()});
        if (it == se.begin() or get<1>(*--it) < i) return -1; // nao tem
        return get<2>(*it);
    }
};
```

2.8. RMQ Sparse Table $< O(n), O(1) >$

TIME	SPACE
build : $O(n)$ query : $O(1)$	$O(n)$

Estructura de **Range Minimum Query** con complejidad $O(n)$ de construcción y $O(1)$ por consulta, basada en la técnica de **Sparse Table + Cartesian Tree / bitmask trick** (método de Fischer-Heun). En caso de haber varios índices con el mismo valor mínimo, `query` retorna el de menor índice.

Funciones principales:

- `op(x, y)`: compara dos índices `x`, `y` y retorna el de menor valor en `v` (o menor índice en caso de empate), sirviendo como operador de comparación base para todo el resto de la estructura.
- `msb(x)`: calcula el bit más significativo de `x` usando `__builtin_clz`, utilizado para operar sobre las máscaras de bits.
- `small(r, sz)`: resuelve en $O(1)$ el mínimo dentro de un bloque de tamaño `sz` (por defecto `b = 30`) terminando en la posición `r`, usando la máscara precomputada `mask[r]`.
- Constructor `rmq(v_)`: divide el arreglo en bloques de tamaño `b`, precomputa para cada posición una máscara (`mask`) que codifica, mediante una pila monótona implícita (bits), el rango de elementos hacia la izquierda que siguen siendo "candidatos" a mínimo local. Luego construye una **sparse table** (`t`) sobre los mínimos de cada bloque, permitiendo responder consultas entre bloques completos en $O(1)$.
- `index_query(l, r)`: si el rango cabe dentro de un solo bloque ($r - l + 1 \leq b$), resuelve directamente con `small`. En caso contrario, combina el mínimo de los bloques parciales en los extremos (resueltos con `small`) con el mínimo de los bloques completos intermedios (resuelto en $O(1)$ vía la **sparse table** `t`).
- `query(l, r)`: retorna el valor mínimo en $v[l..r]$, llamando internamente a `index_query` y accediendo a `v` en la posición resultante.

Es una alternativa más eficiente en tiempo de consulta que la **sparse table** clásica ($O(n \log n)$ de construcción, $O(1)$ de consulta), a costa de mayor complejidad de implementación.

estructuras/rmq_on_o1.cpp

```
// RMQ <O(n), O(1)> - min queue
//
// O(n) pra buildar, query O(1)
// Se tiver varios minimos, retorna
// o de menor indice

template<typename T> struct rmq {
    vector<T> v;
    int n; static const int b = 30;
    vector<int> mask, t;

    int op(int x, int y) { return v[x] <= v[y] ? x : y; }
    int msb(int x) { return __builtin_clz(1)-__builtin_clz(x); }
    int small(int r, int sz = b) { return r-msb(mask[r]&((1<<sz)-1)); }
    rmq() {}
    rmq(const vector<T>& v_) : v(v_), n(v.size()), mask(n), t(n) {
        for (int i = 0, at = 0; i < n; mask[i++] = at |= 1) {
            at = (at<<1)&((1<<b)-1);
            while (at and op(i-msb(at&-at), i) == i) at ^= at&-at;
        }
        for (int i = 0; i < n/b; i++) t[i] = small(b*i+b-1);
        for (int j = 1; (1<<j) <= n/b; j++) for (int i = 0; i+(1<<j) <= n/b; i++)
            t[n/b*j+i] = op(t[n/b*(j-1)+i], t[n/b*(j-1)+i+(1<<(j-1))]);
```

```
    }
    int index_query(int l, int r) {
        if (r-l+1 <= b) return small(r, r-l+1);
        int x = l/b+1, y = r/b-1;
        if (x > y) return op(small(l+b-1), small(r));
        int j = msb(y-x+1);
        int ans = op(small(l+b-1), op(t[n/b*j+x], t[n/b*j+y-(1<<j)+1]));
        return op(ans, small(r));
    }
    T query(int l, int r) { return v[index_query(l, r)]; }
};
```

2.9. Sparse Table (RMQ)

TIME	SPACE
build : $O(n \log n)$ query : $O(1)$	$O(n \log n)$

Estructura clásica de **Sparse Table** para resolver **Range Minimum Query** (RMQ), aprovechando que el operador mínimo es idempotente, lo cual permite responder consultas de rango en $O(1)$ tras un preprocesamiento de $O(n \log n)$. **`MAX2`** debe definirse como $\log(\text{MAX})$, es decir, la cantidad de niveles necesarios para cubrir arreglos de tamaño **`MAX`**.

Funciones principales:

- **`build(n2, v)`**: construye la tabla **`m`** en $O(n \log n)$. La fila base **`m[0]`** se inicializa directamente con los valores del arreglo **`v`**; cada fila siguiente **`m[j]`** se calcula combinando pares de mínimos de la fila anterior **`m[j-1]`**, de modo que **`m[j][i]`** almacena el mínimo del rango $[i, i + 2^j - 1]$.
- **`query(a, b)`**: responde en $O(1)$ el mínimo del rango $[a, b]$. Calcula $j = \text{floor}(\log_2(b - a + 1))$ mediante **`__builtin_clz`**, y combina dos bloques de tamaño 2^j que se solapan pero cubren completamente el rango (**`m[j][a]`** y **`m[j][b - 2^j + 1]`**), aprovechando la idempotencia del mínimo para ignorar el solapamiento.

A diferencia de la variante $< O(n), O(1) >$ (ver **RMQ Sparse Table**), esta versión es más simple de implementar pero tiene mayor costo de construcción y memoria ($O(n \log n)$ en vez de $O(n)$).

estructuras/sparse_table.cpp

```
// Sparse Table
//
// Resolve RMQ
// MAX2 = log(MAX)
//
// Complexidades:
// build - O(n log(n))
// query - O(1)

namespace sparse {
    int m[MAX2][MAX], n;
    void build(int n2, int* v) {
        n = n2;
        for (int i = 0; i < n; i++) m[0][i] = v[i];
        for (int j = 1; (1<<j) <= n; j++) for (int i = 0; i+(1<<j) <= n; i++)
            m[j][i] = min(m[j-1][i], m[j-1][i+(1<<(j-1))]);
    }
    int query(int a, int b) {
        int j = __builtin_clz(1) - __builtin_clz(b-a+1);
        return min(m[j][a], m[j][b-(1<<j)+1]);
    }
}
```

2.10. Sparse Table Disjunta

TIME	SPACE
build : $O(n \log n)$ query : $O(1)$	$O(n \log n)$

Variante de **Sparse Table** que resuelve **Range Query** en $O(1)$ para **cualquier** operación asociativa **`op`** (no necesariamente idempotente como el mínimo), a diferencia de la Sparse Table clásica que solo funciona correctamente con operaciones idempotentes. **`MAX2`** debe definirse como $\log(\text{MAX})$. Internamente, el arreglo se extiende a la potencia de 2 más cercana hacia arriba para simplificar la construcción por bloques disjuntos.

Funciones principales:

- **`op(a, b)`**: operación asociativa a resolver sobre rangos; en este ejemplo es el mínimo, pero puede reemplazarse por cualquier otra operación asociativa (suma, xor, gcd, etc.), siempre que no dependa de idempotencia.
- **`build(n2, v2)`**: construye la tabla en $O(n \log n)$. Extiende **`n`** a la siguiente potencia de 2. Para cada nivel **`j`** (de tamaño de bloque **`len = 2^j`**), y cada punto de corte **`c`** (frontera entre bloques disjuntos consecutivos), calcula prefijos hacia la derecha y sufijos hacia la izquierda de **`op`** a partir de **`c`**, cubriendo así cualquier rango $[l, r]$ que atraviese la frontera **`c`** sin necesidad de combinar rangos solapados.

- `query(l, r)`: responde en $O(1)$ el resultado de `op` sobre el rango $[l, r]$. Si $l = r$, retorna directamente `v[l]`. En otro caso, calcula j como el bit más significativo de l xor r (que identifica el nivel cuya frontera de bloque separa a `l` y `r`), y combina `m[j][l]` (sufijo desde `l` hasta la frontera) con `m[j][r]` (prefijo desde la frontera hasta `r`) mediante `op`.

A diferencia de la Sparse Table clásica (ver **Sparse Table (RMQ)**), que solo es válida para operaciones idempotentes, esta versión "disjunta" es la opción correcta cuando se necesita responder consultas de rango en $O(1)$ para operaciones como suma, xor o gcd.

estructuras/sparse_table_disjunta.cpp

```cpp
// Sparse Table Disjunta
//
// Resolve qualquer operacao associativa
// MAX2 = log(MAX)
//
// Complexidades:
// build - O(n log(n))
// query - O(1)

namespace sparse {
    int m[MAX2][2*MAX], n, v[2*MAX];
    int op(int a, int b) { return min(a, b); }
    void build(int n2, int* v2) {
        n = n2;
        for (int i = 0; i < n; i++) v[i] = v2[i];
        while (n&(n-1)) n++;
        for (int j = 0; (1<<j) < n; j++) {
            int len = 1<<j;
            for (int c = len; c < n; c += 2*len) {
                m[j][c] = v[c], m[j][c-1] = v[c-1];
                for (int i = c+1; i <  c+len; i++) m[j][i] = op(m[j][i-1], v[i]);
                for (int i = c-2; i >= c-len; i--) m[j][i] = op(v[i], m[j][i+1]);
            }
        }
    }
    int query(int l, int r) {
        if (l == r) return v[l];
        int j = __builtin_clz(1) - __builtin_clz(l^r);
        return op(m[j][l], m[j][r]);
    }
}
```

2.11. Mo's Algorithm (com Ordenação de Hilbert)

TIME	SPACE
$O(n\sqrt{q})$	$O(n+q)$

Responde offline un conjunto de q consultas de rango $[l, r]$ sobre un arreglo `v`, reordenando las consultas de forma que el costo total de mover los punteros `l` y `r` entre consultas consecutivas sea $O(n\sqrt{q})$ en total. Para alcanzar esta cota, `SQ` (tamaño de bloque) debe elegirse como $\frac{n}{\sqrt{q}}$. En este ejemplo, `ans` mantiene la cantidad de valores **distintos** dentro del rango actual, pero `insert`/`erase` pueden adaptarse a cualquier otra estadística incremental.

Funciones principales:

- `insert(p)`: agrega la posición `p` al rango actual, incrementando `freq[v[p]]` y sumando a `ans` si ese valor pasó a tener frecuencia exactamente 1 (es decir, si es una nueva aparición distinta).
- `erase(p)`: remueve la posición `p` del rango actual, de forma simétrica a `insert`, restando de `ans` si el valor deja de tener frecuencia 1.
- `hilbert(x, y)`: calcula la distancia a lo largo de una curva de Hilbert para el punto `(x, y)`, usada como clave de ordenamiento alternativa a la clásica ordenación por bloques (`SQ`). En la práctica, ordenar las consultas por su distancia de Hilbert suele dar un mejor rendimiento constante que el orden por bloques tradicional, evitando además el caso patológico de éste último.
- `MO(q)`: función principal. Ordena el vector de consultas `q` (pares `{l, r}`) según la clave elegida (controlada por el macro `HILBERT`): por distancia de Hilbert si `HILBERT` está activo, o por el esquema clásico de bloques con **serpenteo** (alternando dirección según la paridad del bloque, para evitar saltos innecesarios de `r`) en caso contrario. Luego procesa las consultas en el orden resultante, moviendo los punteros `l`/`r` con `insert`/`erase` hasta alcanzar cada rango, y guarda `ans` en la posición original de cada consulta (`ret[i]`).

El macro `HILBERT` permite alternar fácilmente entre ambas estrategias de ordenamiento sin reescribir el resto del algoritmo; en competencia suele preferirse Hilbert quando existe riesgo de que el patrón de consultas ataque el peor caso del ordenamiento por bloques.

estructuras/mo_algorithm.cpp

```cpp
// MO
```

```
//
// Para ter o bound abaixo, escolher
// SQ = n / sqrt(q)
//
// O(n * sqrt(q))

const int MAX = 1e5+10;
const int SQ = sqrt(MAX);
int v[MAX];

int ans, freq[MAX];

inline void insert(int p) {
    int o = v[p];
    freq[o]++;
    ans += (freq[o] == 1);
}

inline void erase(int p) {
    int o = v[p];
    ans -= (freq[o] == 1);
    freq[o]--;
}

inline ll hilbert(int x, int y) {
    static int N = 1 << (__builtin_clz(0) - __builtin_clz(MAX));
    int rx, ry, s;
    ll d = 0;
    for (s = N/2; s > 0; s /= 2) {
        rx = (x & s) > 0, ry = (y & s) > 0;
        d += s * ll(s) * ((3 * rx) ^ ry);
        if (ry == 0) {
            if (rx == 1) x = N-1 - x, y = N-1 - y;
            swap(x, y);
        }
    }
    return d;
}

#define HILBERT true
vector<int> MO(vector<pair<int, int>> &q) {
    ans = 0;
    int m = q.size();
    vector<int> ord(m);
    iota(ord.begin(), ord.end(), 0);
#if HILBERT
    vector<ll> h(m);
    for (int i = 0; i < m; i++) h[i] = hilbert(q[i].first,
    q[i].second);
    sort(ord.begin(), ord.end(), [&](int l, int r) { return h[l] <
    h[r]; });
#else
    sort(ord.begin(), ord.end(), [&](int l, int r) {
        if (q[l].first / SQ != q[r].first / SQ) return q[l].first <
        q[r].first;
        if ((q[l].first / SQ) % 2) return q[l].second > q[r].second;
        return q[l].second < q[r].second;
    });
#endif
    vector<int> ret(m);
    int l = 0, r = -1;

    for (int i : ord) {
        int ql, qr;
        tie(ql, qr) = q[i];
        while (r < qr) insert(++r);
        while (l > ql) insert(--l);
        while (l < ql) erase(l++);
        while (r > qr) erase(r--);
        ret[i] = ans;
    }
    return ret;
}
```

3. Ordenamiento y Búsqueda

3.1. Two Pointers

Some uses:

Subarray sum: Given an array of n positive integers and a target sum x, and we want to find a subarray whose sum is x or report that there is no such subarray.

2SUM Problem: Given an array of n numbers and a target sum x, find two array values such that their sum is x, or report that no such values exist.

Number of Smaller: We have two arrays a and b. We want to calculate for each element b_j how many such i exist that $a_i < b_j$.

```
i = 0
for j = 0..b.size()-1:
  while i < a.size() && a[i] < b[j]:
      i++
  res[j] = i
```

3.2. Búsqueda binaria en funciones monótonas

Encuentra la máxima x tal que $f(x) = \text{true}$

TIME
$O(\log n)$

ordenamiento_busqueda/last_true.cpp

```cpp
int last_true(int lo, int hi, function<bool(int)> f) {
    // if none of the values in the range work, return lo - 1
    lo--;
    while (lo < hi) {
        // find the middle of the current range (rounding up)
        int mid = lo + (hi - lo + 1) / 2;
        if (f(mid)) {
            // if mid works, then all numbers smaller than mid also work
            lo = mid;
        } else {
            // if mid does not work, greater values would not work either
            hi = mid - 1;
        }
    }
    return lo;
}
```

Encuentra la mínima x tal que $f(x) = \text{true}$

TIME
$O(\log n)$

ordenamiento_busqueda/first_true.cpp

```cpp
int first_true(int lo, int hi, function<bool(int)> f) {
    hi++;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (f(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}
```

4. Teoría de gráficas

4.1. Graph Traversal

4.1.1. Floodfill

Flood fill is an algorithm that identifies and labels the connected component that a particular cell belongs to in a multidimensional array.

> ⚠ **Advertencia**
>
> Recursive implementations of flood fill sometimes lead to
> - Memory limit exceeded if you run the recursive implementation on a really big grid (i.e., running the above code on a 4000×4000 grid may exceed 256 MB)
>   - Non-recursive implementations generally use less memory than recursive ones.

Example (recursive implementation)

graficas/floodfill_recursive.cpp

```cpp
const int MAX_N = 1000;

int grid[MAX_N][MAX_N];  // the grid itself
int row_num;
int col_num;
bool visited[MAX_N][MAX_N];  // keeps track of which nodes have been visited
```

```
int curr_size = 0;          // reset to 0 each time we start a new component

void floodfill(int r, int c, int color) {
    if ((r < 0 || r >= row_num || c < 0 || c >= col_num)  // if out of bounds
        || grid[r][c] != color                            // wrong color
        || visited[r][c]                                  // already visited this square
    )
        return;

    visited[r][c] = true;  // mark current square as visited
    curr_size++;           // increment the size for each square we visit

    // recursively call flood fill for neighboring squares
    floodfill(r, c + 1, color);
    floodfill(r, c - 1, color);
    floodfill(r - 1, c, color);
    floodfill(r + 1, c, color);
}

int main() {
    /*
     * input code and other problem-specific stuff here
     */
    for (int i = 0; i < row_num; i++) {
        for (int j = 0; j < col_num; j++) {
            if (!visited[i][j]) {
                curr_size = 0;
                /*
                 * start a flood fill if the square hasn't already been visited,
                 * and then store or otherwise use the component size
                 * for whatever it's needed for
                 */
                floodfill(i, j, grid[i][j]);
            }
        }
    }
    return 0;
}
```

Example (iterative implementation)

graficas/floodfill_iterative.cpp

```
#include <bits/stdc++.h>

using namespace std;

const int MAX_N = 2500;
const int R_CHANGE[]{0, 1, 0, -1};
const int C_CHANGE[]{1, 0, -1, 0};

int row_num;
int col_num;
string building[MAX_N];
bool visited[MAX_N][MAX_N];

void floodfill(int r, int c) {
    // Note: you can also use a queue and pop from the front for a BFS-based
    // approach
    stack<pair<int, int>> frontier;
    frontier.push({r, c});
    while (!frontier.empty()) {
        r = frontier.top().first;
        c = frontier.top().second;
        frontier.pop();

        if (r < 0 || r >= row_num || c < 0 || c >= col_num ||
        building[r][c] == '#' ||
            visited[r][c])
            continue;

        visited[r][c] = true;
        for (int i = 0; i < 4; i++) {
            frontier.push({r + R_CHANGE[i], c + C_CHANGE[i]});
        }
    }
}
```

```
34
35 int main() {
36     cin >> row_num >> col_num;
37     for (int i = 0; i < row_num; i++) { cin >> building[i]; }
38
39     int room_num = 0;
40     for (int i = 0; i < row_num; i++) {
41         for (int j = 0; j < col_num; j++) {
42             if (building[i][j] == '.' && !visited[i][j]) {
43                 floodfill(i, j);
44                 room_num++;
45             }
46         }
47     }
48     cout << room_num << endl;
49 }
```

4.2. Disjoin Set Union

Disjoint Set Union

TIME

- `find`: $O(\log n)$
- Con path compression y union by size/rank: $O(\alpha(n))$, donde α es la inversa de la función de Ackermann (amortizada).
- Con union by size/rank, peros sin path compression: $O(\log n)$ por consulta.

graficas/dsu.cpp

```
1  #include "setup/template.h"
2
3  struct DSU {
4      vector<int> parent, rank, size;
5      int components;
6
7      DSU(int n) : parent(n + 1), rank(n + 1, 0), size(n + 1, 1),
       components(n) {
8          iota(all(parent), 0);
9      }
10
11     // Retorna el representante de la componente de "x"
12     int find(int x) {
13         return parent[x] = (parent[x] == x ? x : find(parent[x]));
14     }
15
16     // Retorna si "x" y "y" se encuentran en la misma componente
17     bool connected(int x, int y) {
18         return find(x) == find(y);
19     }
20
21     // Retorna el tamaño de la componente de "x"
22     int get_size(int x) {
23         return size[find(x)];
24     }
25
26     // Retorna la cantidad de componentes
27     int count() {
28         return components;
29     }
30
31     // Une dos componentes y retorna el nuevo representante o retorna -1
       si ya son parte de la misma componente
32     int merge(int x, int y) {
33         x = find(x);
34         y = find(y);
35
36         if (x == y) return -1;
37
38         components--;
39
40         if (rank[x] > rank[y]) swap(x, y);
41         parent[x] = y;
42         size[y] += size[x];
43
44         if (rank[x] == rank[y]) rank[y]++;
45         return y;
46     }
47 };
```

DSU (Union-Find) - Variantes

TIME	SPACE
$O(\alpha(n)) \sim O(\log n)$	$O(n)$

Colección de implementaciones de Disjoint Set Union (DSU), estructura que une conjuntos disjuntos y determina a qué conjunto pertenece un elemento mediante su id. Incluye las siguientes variantes:

- **DSU básico**: unión por tamaño (`unite`) con compresión de caminos (`find`), logrando complejidad amortizada prácticamente constante $O(\alpha(n))$. Base para componentes conexas, Kruskal (MST) y detección de ciclos.
- **DSU bipartido**: además de unir vértices y encontrar componentes (`find`, `unite`), mantiene coloración (`color`) para saber si la componente de un vértice sigue siendo bipartita (`bip`) tras cada unión. Útil para verificar bipartición dinámica de un grafo procesando aristas en línea.
- **DSU persistente**: cada unión se marca con un timestamp `t` estrictamente creciente. Permite consultar conectividad (`find`) en cualquier instante pasado sin deshacer operaciones; útil para consultas offline sobre el historial de uniones.
- **DSU con rollback**: omite compresión de caminos para mantener reversibilidad. `checkpoint()` guarda el estado en $O(1)$ y `rollback()` revierte los cambios desde el último checkpoint en $O(m)$, donde m es el número de modificaciones registradas mediante `save`. Usado en técnicas offline como **divide and conquer** sobre el tiempo (**offline dynamic connectivity**).

graficas/dsu2.cpp

```cpp
// DSU
//
// Une dois conjuntos e acha a qual conjunto um elemento pertence por seu id
//
// find e unite: O(a(n)) ~= O(1) amortizado

struct dsu {
    vector<int> id, sz;

    dsu(int n) : id(n), sz(n, 1) { iota(id.begin(), id.end(), 0); }

    int find(int a) { return a == id[a] ? a : id[a] = find(id[a]); }

    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        sz[a] += sz[b], id[b] = a;
    }
};

// DSU de bipartido
//
// Une dois vertices e acha a qual componente um vertice pertence
// Informa se a componente de um vertice e bipartida
//
// find e unite: O(log(n))

struct dsu {
    vector<int> id, sz, bip, c;

    dsu(int n) : id(n), sz(n, 1), bip(n, 1), c(n) {
        iota(id.begin(), id.end(), 0);
    }

    int find(int a) { return a == id[a] ? a : find(id[a]); }
    int color(int a) { return a == id[a] ? c[a] : c[a] ^ color(id[a]); }

    void unite(int a, int b) {
        bool change = color(a) == color(b);
        a = find(a), b = find(b);
        if (a == b) {
            if (change) bip[a] = 0;
            return;
        }

        if (sz[a] < sz[b]) swap(a, b);
        if (change) c[b] = 1;
        sz[a] += sz[b], id[b] = a, bip[a] &= bip[b];
    }
};


// DSU Persistente
//
// Persistencia parcial, ou seja, tem que ir
// incrementando o 't' no une
//
// find e unite: O(log(n))

```

```cpp
struct dsu {
    vector<int> id, sz, ti;

    dsu(int n) : id(n), sz(n, 1), ti(n, -INF) {
        iota(id.begin(), id.end(), 0);
    }

    int find(int a, int t) {
        if (id[a] == a or ti[a] > t) return a;
        return find(id[a], t);
    }

    void unite(int a, int b, int t) {
        a = find(a, t), b = find(b, t);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        sz[a] += sz[b], id[b] = a, ti[b] = t;
    }
};

// DSU com rollback
//
// checkpoint(): salva o estado atual de todas as variaveis
// rollback(): retorna para o valor das variaveis para
// o ultimo checkpoint
//
// Sempre que uma variavel muda de valor, adiciona na stack
//
// find e unite: O(log(n))
// checkpoint: O(1)
// rollback: O(m) em que m e o numero de vezes que alguma
// variavel mudou de valor desde o ultimo checkpoint

struct dsu {
    vector<int> id, sz;
    stack<stack<pair<int&, int>>> st;

    dsu(int n) : id(n), sz(n, 1) {
        iota(id.begin(), id.end(), 0), st.emplace();
    }

    void save(int &x) { st.top().emplace(x, x); }

    void checkpoint() { st.emplace(); }

    void rollback() {
        while(st.top().size()) {
            auto [end, val] = st.top().top(); st.top().pop();
            end = val;
        }
        st.pop();
    }

    int find(int a) { return a == id[a] ? a : find(id[a]); }

    void unite(int a, int b) {
        a = find(a), b = find(b);
        if (a == b) return;
        if (sz[a] < sz[b]) swap(a, b);
        save(sz[a]), save(id[b]);
        sz[a] += sz[b], id[b] = a;
    }
};
```

4.3. Small-to-Large Merging (Sack / DSU on Tree)

TIME	SPACE
$O(n \log n)$	$O(n)$

Técnica de **DSU on Tree** (también conocida como **Sack**) que responde de forma **offline** consultas sobre todas las sub-árboles de un árbol enraizado, evitando el costo de $O(n^2)$ que resultaría de recalcular desde cero la información de cada sub-árbol. En este ejemplo se mantiene `cnt[c]`, la cantidad de veces que aparece el color `c` dentro de la sub-árbol del vértice actual, pero la técnica generaliza a cualquier estadística agregable de forma incremental.

Funciones principales:

- `build(k, d)`: precomputa el tamaño (`sz`) de cada sub-árbol mediante un DFS, y reordena los hijos de `k` para que el hijo de mayor tamaño (**heavy child**) quede siempre en la posición `g[k][0]`. Este reordenamiento es lo que permite luego "saltarlo" eficientemente en `compute`.
- `compute(k, x, dont)`: agrega (`x = 1`) o remueve (`x = -1`) la contribución de todo el sub-árbol de `k` sobre `cnt`, recorriéndolo completo. El parámetro `dont` (activado

por defecto) hace que se salte el primer hijo (`g[k][0]`, el **heavy child**), ya que su información se maneja por separado para lograr la complejidad amortizada $O(n \log n)$.

- `solve(k, keep)`: resuelve recursivamente todos los hijos livianos (**light children**) primero sin conservar su información (`compute` seguido de limpieza), y por último resuelve el **heavy child** conservando su información (gracias al orden de recorrido `!i`, que asegura que el hijo pesado —índice 0— se procese al final con `keep = 1`). Luego agrega la contribución del propio `k` y de sus hijos livianos mediante `compute(k, 1)`. En este punto, `cnt[i]` refleja exactamente cuántas veces aparece el color `i` en la sub-árbol de `k`, momento ideal para responder cualquier consulta asociada a `k`. Finalmente, si `keep = 0` (es decir, `k` es un hijo liviano de su padre), se revierte la contribución con `compute(k, -1, 0)` para no contaminar el conteo de sub-árboles hermanas.

La clave de la complejidad $O(n \log n)$ es que cada vértice es visitado como parte de un "hijo liviano que se descarta" a lo sumo $O(\log n)$ veces a lo largo de todo el algoritmo (cada vez que esto ocurre, el tamaño de su componente al menos se duplica).

graficas/sack.cpp

```cpp
// Sack (DSU em arvores)
//
// Responde queries de todas as sub-arvores
// offline
//
// O(n log(n))

int sz[MAX], cor[MAX], cnt[MAX];
vector<int> g[MAX];

void build(int k, int d=0) {
    sz[k] = 1;
    for (auto& i : g[k]) {
        build(i, d+1); sz[k] += sz[i];
        if (sz[i] > sz[g[k][0]]) swap(i, g[k][0]);
    }
}

void compute(int k, int x, bool dont=1) {
    cnt[cor[k]] += x;
    for (int i = dont; i < g[k].size(); i++)
        compute(g[k][i], x, 0);
}

void solve(int k, bool keep=0) {
    for (int i = int(g[k].size())-1; i >= 0; i--)
        solve(g[k][i], !i);
    compute(k, 1);

    // agora cnt[i] tem quantas vezes a cor
    // i aparece na sub-arvore do k

    if (!keep) compute(k, -1, 0);
}
```

4.4. Topological Sort

A topological sort of a directed acyclic graph is a linear ordering of its vertices such that for every directed edge $u \to v$ from vertex u to vertex v, u comes before v in the ordering.

DFS Version

graficas/topological_sort_dfs.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

vector<int> top_sort;
vector<vector<int>> graph;
vector<bool> visited;

void dfs(int node) {
    for (int next : graph[node]) {
        if (!visited[next]) {
            visited[next] = true;
            dfs(next);
        }
    }
    top_sort.push_back(node);
}

```

```cpp
int main() {
    int n, m;  // The number of nodes and edges respectively
    cin >> n >> m;

    graph = vector<vector<int>>(n);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        graph[a - 1].push_back(b - 1);
    }

    visited = vector<bool>(n);
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            visited[i] = true;
            dfs(i);
        }
    }
    reverse(top_sort.begin(), top_sort.end());

    vector<int> ind(n);
    for (int i = 0; i < n; i++) { ind[top_sort[i]] = i; }

    // Check if the topological sort is valid
    bool valid = true;
    for (int i = 0; i < n; i++) {
        for (int j : graph[i]) {
            if (ind[j] <= ind[i]) {
                valid = false;
                goto answer;
            }
        }
    }
answer:;

    if (valid) {
        for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 << ' '; }
        cout << top_sort.back() + 1 << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
}
```

BFS Version (Kahn's Algorithm)

graficas/topological_sort_bfs.cpp

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
    int n, m;
    cin >> n >> m;

    vector<vector<int>> graph(n);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        graph[a - 1].push_back(b - 1);
    }

    vector<int> in_degree(n);
    for (const vector<int> &nodes : graph) {
        for (int node : nodes) { in_degree[node]++; }
    }

    queue<int> queue;
    for (int i = 0; i < n; i++) {
        if (in_degree[i] == 0) { queue.push(i); }
    }
    vector<int> top_sort;
    while (!queue.empty()) {
        int curr = queue.front();
        queue.pop();
        top_sort.push_back(curr);
        for (int next : graph[curr]) {
            if (--in_degree[next] == 0) { queue.push(next); }
        }
    }

```

```
    if (top_sort.size() == n) {
        for (int i = 0; i < n - 1; i++) { cout << top_sort[i] + 1 << ' '; }
        cout << top_sort.back() + 1 << endl;
    } else {
        cout << "IMPOSSIBLE" << endl;
    }
}
```

4.5. Detección de ciclos

Implementación para gráficas dirigidas

graficas/deteccion_ciclos_dirigidas.cpp

```
int n;
vector<vector<int>> adj;
vector<char> color;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v) {
    color[v] = 1;
    for (int u : adj[v]) {
        if (color[u] == 0) {
            parent[u] = v;
            if (dfs(u))
                return true;
        } else if (color[u] == 1) {
            cycle_end = v;
            cycle_start = u;
            return true;
        }
    }
    color[v] = 2;
    return false;
}

void find_cycle() {
    color.assign(n, 0);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (color[v] == 0 && dfs(v))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);
        reverse(cycle.begin(), cycle.end());

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

Implementación para gráficas no dirigidas

graficas/deteccion_ciclos_no_dirigidas.cpp

```
int n;
vector<vector<int>> adj;
vector<bool> visited;
vector<int> parent;
int cycle_start, cycle_end;

bool dfs(int v, int par) { // passing vertex and its parent vertex
    visited[v] = true;
    for (int u : adj[v]) {
        if(u == par) continue; // skipping edge to parent vertex
        if (visited[u]) {
            cycle_end = v;
            cycle_start = u;
```

```
            return true;
        }
        parent[u] = v;
        if (dfs(u, parent[u]))
            return true;
    }
    return false;
}

void find_cycle() {
    visited.assign(n, false);
    parent.assign(n, -1);
    cycle_start = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && dfs(v, parent[v]))
            break;
    }

    if (cycle_start == -1) {
        cout << "Acyclic" << endl;
    } else {
        vector<int> cycle;
        cycle.push_back(cycle_start);
        for (int v = cycle_end; v != cycle_start; v = parent[v])
            cycle.push_back(v);
        cycle.push_back(cycle_start);

        cout << "Cycle found: ";
        for (int v : cycle)
            cout << v << " ";
        cout << endl;
    }
}
```

4.6. Detección de Ciclo (Tortoise and Hare)

TIME	SPACE
$O(\mu + \lambda)$	$O(1)$

Algoritmo de Floyd (**Tortoise and Hare**) para detectar el ciclo de una función iterada f aplicada sobre un valor inicial **`f0`**, usando únicamente $O(1)$ de memoria. Retorna un par **`{st, len}`**: **`st`** es la cantidad de pasos necesarios desde **`f0`** hasta el inicio del ciclo (la "cola"), y **`len`** es la longitud del propio ciclo. La complejidad temporal es lineal respecto a cuánto hay que avanzar para empezar a ciclar (usualmente denotado $\mu + \lambda$, donde μ es la longitud de la cola y λ la del ciclo).

Funcionamiento (**`find_cycle()`**), en tres fases:

- **Detección de colisión**: se avanza **`tort`** (tortuga) un paso a la vez y **`hare`** (liebre) dos pasos a la vez, contando **`t`**, hasta que ambos coincidan; esto garantiza que existe un ciclo y que se encuentran dentro de él.
- **Localización del inicio del ciclo**: se reinicia **`tort`** a **`f0`** y se avanza tanto **`tort`** como **`hare`** un paso a la vez simultáneamente; el punto donde vuelven a coincidir es exactamente el inicio del ciclo, y la cantidad de pasos dados (**`st`**) es la longitud de la cola.
- **Cálculo de la longitud del ciclo**: desde el punto de encuentro (**`tort`**, ya en el inicio del ciclo), se avanza **`hare`** repetidamente hasta volver a **`tort`**, contando los pasos (**`len`**), lo cual da la longitud exacta del ciclo.

Es útil en problemas sobre **functional graphs** o secuencias generadas iterativamente (por ejemplo, secuencias pseudoaleatorias, iteración de funciones numéricas) donde se necesita conocer la estructura de "cola + ciclo" sin poder almacenar todo el historial visitado, a diferencia de un enfoque con **`set`**/**`map`** que usaría $O(\mu + \lambda)$ de memoria.

graficas/tortoise_hare.cpp

```
// Deteccao de ciclo - Tortoise and Hare
//
// Linear no tanto que tem que andar pra ciclar,
// O(1) de memoria
// Retorna um par com o tanto que tem que andar
// do f0 ate o inicio do ciclo e o tam do ciclo

pair<ll, ll> find_cycle() {
    ll tort = f(f0);
    ll hare = f(f(f0));
    ll t = 0;
    while (tort != hare) {
        tort = f(tort);
        hare = f(f(hare));
        t++;
    }
```

```
    ll st = 0;
    tort = f0;
    while (tort != hare) {
        tort = f(tort);
        hare = f(hare);
        st++;
    }

    ll len = 1;
    hare = f(tort);
    while (tort != hare) {
        hare = f(hare);
        len++;
    }
    return {st, len};
}
```

4.7. Lowest Common Ancestor (LCA)

Lowest Common Ancestor - Binary Lifting

graficas/lca.cpp

```cpp
#include<bits/stdc++.h>
using namespace std;

// Cantidad de nodos del árbol más un margen para árboles 1-indexados
const int N = 3e5 + 9;
// Cantidad máxima de niveles ceil(log2(#nodos))
const int LG = 18;

// adj[u] = lista de adyacencia del nodo u
vector<int> adj[N];
// up[i][j] = 2^j-ésimo ancestro de i
int up[N][LG + 1];
// depth[i] = profundidad del nodo i
int depth[N];
// subtree_size[i] = tamaño del subárbol con raíz en i
int subtree_size[N];

void dfs(int u, int p = 0) {
    up[u][0] = p;
    depth[u] = depth[p] + 1;
    subtree_size[u] = 1;

    for (int i = 1; i <= LG; i++) {
        up[u][i] = up[up[u][i - 1]][i - 1];
    }

    for (auto v: adj[u]) {
        if (v != p) {
            dfs(v, u);
            subtree_size[u] += subtree_size[v];
        }
    }
}

// LCA de u y v
int lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);

    for (int k = LG; k >= 0; k--) {
        if (depth[up[u][k]] >= depth[v]) u = up[u][k];
    }

    if (u == v) return u;

    for (int k = LG; k >= 0; k--) {
        if (up[u][k] != up[v][k]) u = up[u][k], v = up[v][k];
    }

    return up[u][0];
}

// k-ésimo ancestro de u, k >= 0
int kth(int u, int k) {
    assert(k >= 0);
    for (int i = 0; i <= LG; i++){
        if (k & (1 << i)) u = up[u][i];
    }
    return u;
}
```

```cpp

// Distancia entre u y v
int dist(int u, int v) {
    int l = lca(u, v);
    return depth[u] + depth[v] - (depth[l] << 1);
}

// k-ésimo nodo en el camino de u a v, el nodo 0 es u
int go(int u, int v, int k) {
    int l = lca(u, v);
    int d = depth[u] + depth[v] - (depth[l] << 1);
    assert(k <= d);
    if (depth[l] + k <= depth[u]) return kth(u, k);
    k -= depth[u] - depth[l];
    return kth(v, depth[v] - depth[l] - k);
}
```

4.8. Centro y Diámetro de un Árbol

TIME	SPACE
$O(n)$	$O(n)$

Encuentra el diámetro de un árbol y su(s) centro(s) en $O(n)$. Un árbol siempre tiene uno o dos centros, y estos se ubican exactamente en el punto medio del camino que forma el diámetro. Es útil como paso previo para técnicas de **centroid decomposition**, **rerooting**, o para acotar la altura de un árbol al elegir la raíz óptima (minimizando la excentricidad máxima).

Funciones principales:

- **`dfs(v)`**: recorre el árbol desde `v` calculando profundidades (`d`) y padres (`par`), actualizando en variables externas (`f`, `df`) el vértice más lejano encontrado hasta el momento y su distancia.
- **`center()`**: aplica la técnica clásica de "doble BFS/DFS" para hallar el diámetro: primero corre `dfs` desde un vértice arbitrario (el 0) para encontrar el vértice `f` más lejano (un extremo del diámetro); luego corre `dfs` nuevamente desde `f` (ahora usado como `root`) para encontrar el otro extremo del diámetro y su longitud `df`. Finalmente, reconstruye el camino del diámetro siguiendo los punteros `par` desde el segundo extremo hasta la raíz, y selecciona como centro(s) el o los vértices ubicados en la posición $\frac{\text{df}}{2}$ (y $\frac{\text{df}+1}{2}$ si el diámetro es impar, dando dos centros). Retorna el par `{diámetro, vector de centros}`.

Nota: el código asume que el árbol contiene al vértice `0` y reinicia `par[root] = -1` antes de la segunda pasada; si se reutiliza `center()` sobre múltiples árboles o componentes, hay que asegurarse de resetear `d`/`par` apropiadamente entre llamadas.

graficas/centro_arvore.cpp

```cpp
// Centro de arvore
//
// Retorna o diametro e o(s) centro(s) da arvore
// Uma arvore tem sempre um ou dois centros e estes estao no meio do diametro
//
// O(n)

vector<int> g[MAX];
int d[MAX], par[MAX];

pair<int, vector<int>> center() {
    int f, df;
    function<void(int)> dfs = [&] (int v) {
        if (d[v] > df) f = v, df = d[v];
        for (int u : g[v]) if (u != par[v])
            d[u] = d[v] + 1, par[u] = v, dfs(u);
    };

    f = df = par[0] = -1, d[0] = 0;
    dfs(0);
    int root = f;
    f = df = par[root] = -1, d[root] = 0;
    dfs(root);

    vector<int> c;
    while (f != -1) {
        if (d[f] == df/2 or d[f] == (df+1)/2) c.push_back(f);
        f = par[f];
    }

    return {df, c};
}
```

4.9. Euler Tour (Intervalos de Entrada/Salida)

TIME	SPACE
$O(n)$	$O(n)$

Recorre un árbol mediante DFS asignando a cada vértice un intervalo `[start[v], end[v])` que representa el rango de tiempos (marcados por un contador global `timer`) durante el cual `v` y toda su sub-árbol permanecen "activos" en el recorrido. Esta técnica traduce consultas sobre sub-árboles a consultas sobre **rangos contiguos** de un arreglo, permitiendo resolverlas con estructuras de datos de rango como **Segment Tree** o **Fenwick Tree** (BIT).

Funcionamiento (`euler_tour(at, prev)`):

- Al entrar al vértice `at`, se registra `start[at] = timer` y se incrementa `timer`, marcando el inicio de su intervalo.
- Se recorre recursivamente cada vecino `n` de `at` (excluyendo `prev`, el padre, para no retroceder en un árbol no dirigido representado como grafo).
- Al salir de `at` (tras procesar todos sus hijos), se registra `end[at] = timer`, marcando el final de su intervalo. Todo vértice `u` que sea descendiente de `at` cumplirá $\text{start}[\text{at}] \leq \text{start}[u] < \text{end}[\text{at}]$, propiedad clave que permite identificar la sub-árbol de `at` como el rango contiguo $[\text{start}[\text{at}], \text{end}[\text{at}])$.

Es la base para técnicas como **Heavy-Light Decomposition**, consultas de subárbol con **Segment Tree** / **BIT** (por ejemplo, "sumar x a todo un subárbol" o "consultar el valor de un vértice"), y el cálculo de **Lowest Common Ancestor** (LCA) mediante **Sparse Table** sobre el recorrido de Euler.

graficas/euler_tour.cpp

```cpp
// The graph represented as an adjacency list (0-indexed)
vector<vector<int>> neighbors{{1, 2}, {0}, {0, 3, 4}, {2}, {2}};
vector<int> start(neighbors.size());
vector<int> end(neighbors.size());
int timer = 0;

void euler_tour(int at, int prev) {
    start[at] = timer++;
    for (int n : neighbors[at]) {
        if (n != prev) { euler_tour(n, at); }
    }
    end[at] = timer;
}
```

4.10. Functional Graph (Binary Lifting sobre Ciclos)

TIME	SPACE
build : $O(n)$ f_k : $O(\log(\min(n, k)))$ path : $O(\log(\min(n, k)))$	$O(n)$

Estructura para **functional graphs** (grafos donde cada vértice `i` tiene exactamente una arista saliente hacia `f[i]`), que permite responder en $O(\log(\min(n, k)))$ a dónde se llega tras seguir k aristas desde un vértice, y cuál es el costo acumulado (bajo una operación asociativa `op`, suma por defecto) de recorrer esas k aristas. Todo vértice eventualmente cae en un ciclo; la estructura detecta estos ciclos y combina **binary lifting** (para la parte de "cola" antes del ciclo) con una **segment tree circular** (para moverse dentro del ciclo).

Arreglos principales:

- `rt[i]`: ID de la raíz (índice del ciclo) asociado al vértice `i`.
- `d[i]`: profundidad de `i` respecto al ciclo (0 si `i` está en el ciclo).
- `pos[i]`: posición de `i` dentro del arreglo que concatena todos los ciclos (usado para indexar la segment tree circular `seg`).
- `p[i]`, `pp[i]`, `jmp[i]`: estructura de **binary lifting** irregular: `p[i]` es el padre inmediato, `pp[i]` es un salto más largo elegido dinámicamente para balancear la profundidad, y `jmp[i]` acumula `op` sobre ese salto.

Funciones principales:

- `op(a, b)`: operación asociativa a acumular a lo largo de las aristas recorridas; por defecto es suma, pero puede reemplazarse fácilmente en el código.
- `dfs(i, t)`: recorre el grafo detectando ciclos (mediante los tiempos de visita `vis`) y construyendo, para cada vértice fuera de un ciclo, los punteros de **binary lifting** (`p`, `pp`, `jmp`) de forma que la profundidad se reduzca aproximadamente a la mitad en cada salto largo, garantizando $O(\log n)$ saltos en el peor caso.
- `build(f_, val_)`: inicializa la estructura a partir del vector de funciones `f_` (arista de cada vértice) y opcionalmente un vector de costos `val_` (si se omite, se usa `f_` como costo). Recolecta todos los ciclos encontrados y construye sobre ellos una segment tree circular (`seg`) para responder consultas de suma dentro del ciclo en $O(\log n)$.
- `f_k(i, k)`: retorna el vértice al que se llega partiendo de `i` y siguiendo exactamente `k` aristas. Primero sube usando los saltos de **binary lifting** mientras `i` no esté en el ciclo, y si aún quedan pasos, calcula la posición final dentro del ciclo mediante aritmética modular sobre `pos`.

- **`path(i, k)`**: análogo a **`f_k`**, pero retorna el resultado acumulado de **`op`** a lo largo del camino de **`k`** aristas, en lugar del vértice destino. Dentro del ciclo, usa la segment tree circular para sumar vueltas completas y el tramo parcial restante.

graficas/functional_graph.cpp

```cpp
// Functional Graph
//
// rt[i] fala o ID da raiz associada ao vertice i
// d[i] fala a profundidade (0 sse ta no ciclo)
// pos[i] fala a posicao de i no array que eh a concat. dos ciclos
// build(f, val) recebe a funcao f e o custo de ir de
// i para f[i] (por default, val = f)
// f_k(i, k) fala onde i vai parar se seguir k arestas
// path(i, k) fala o custo (soma) seguir k arestas a partir de i
// Se quiser outra operacao, da pra alterar facil o codigo
// Codigo um pouco louco, tenho que admitir
//
// build - O(n)
// f_k   - O(log(min(n, k)))
// path  - O(log(min(n, k)))

namespace func_graph {
    int n;
    int f[MAX], vis[MAX], d[MAX];
    int p[MAX], pp[MAX], rt[MAX], pos[MAX];
    int sz[MAX], comp;
    vector<vector<int>> ciclo;
    ll val[MAX], jmp[MAX], seg[2*MAX];

    ll op(ll a, ll b) { return a+b; }; // mudar a operacao aqui
    void dfs(int i, int t = 2) {
        vis[i] = t;
        if (vis[f[i]] >= 2) { // comeca ciclo - f[i] eh o rep.
            d[i] = 0, rt[i] = comp;
            sz[comp] = t - vis[f[i]] + 1;
            p[i] = pp[i] = i, jmp[i] = val[i];
            ciclo.emplace_back();
            ciclo.back().push_back(i);
        } else {
            if (!vis[f[i]]) dfs(f[i], t+1);
            rt[i] = rt[f[i]];
            if (sz[comp]+1) { // to no ciclo
                d[i] = 0;
                p[i] = pp[i] = i, jmp[i] = val[i];
                ciclo.back().push_back(i);
            } else { // nao to no ciclo
                d[i] = d[f[i]]+1, p[i] = f[i];
                pp[i] = 2*d[pp[f[i]]] == d[pp[pp[f[i]]]]+d[f[i]] ?
                    pp[pp[f[i]]] : f[i];
                jmp[i] = pp[i] == f[i] ? val[i] : op(val[i],
                    op(jmp[f[i]], jmp[pp[f[i]]]));
            }
        }
        if (f[ciclo[rt[i]][0]] == i) comp++; // fim do ciclo
        vis[i] = 1;
    }
    void build(vector<int> f_, vector<int> val_ = {}) {
        n = f_.size(), comp = 0;
        if (!val_.size()) val_ = f_;
        for (int i = 0; i < n; i++)
            f[i] = f_[i], val[i] = val_[i], vis[i] = 0, sz[i] = -1;

        ciclo.clear();
        for (int i = 0; i < n; i++) if (!vis[i]) dfs(i);
        int t = 0;
        for (auto& c : ciclo) {
            reverse(c.begin(), c.end());
            for (int j : c) {
                pos[j] = t;
                seg[n+t] = val[j];
                t++;
            }
        }
        for (int i = n-1; i; i--) seg[i] = op(seg[2*i], seg[2*i+1]);
    }

    int f_k(int i, ll k) {
        while (d[i] and k) {
            int big = d[i] - d[pp[i]];
            if (big <= k) k -= big, i = pp[i];
            else k--, i = p[i];
```

```
        }
        if (!k) return i;
        return ciclo[rt[i]][(pos[i] - pos[ciclo[rt[i]][0]] + k) %
        sz[rt[i]]];
    }
    ll path(int i, ll k) {
        auto query = [&](int l, int r) {
            ll q = 0;
            for (l += n, r += n; l <= r; ++l/=2, --r/=2) {
                if (l%2 == 1) q = op(q, seg[l]);
                if (r%2 == 0) q = op(q, seg[r]);
            }
            return q;
        };
        ll ret = 0;
        while (d[i] and k) {
            int big = d[i] - d[pp[i]];
            if (big <= k) k -= big, ret = op(ret, jmp[i]), i = pp[i];
            else k--, ret = op(ret, val[i]), i = p[i];
        }
        if (!k) return ret;
        int first = pos[ciclo[rt[i]][0]], last =
        pos[ciclo[rt[i]].back()];

        // k/sz[rt[i]] voltas completas
        if (k/sz[rt[i]]) ret = op(ret, k/sz[rt[i]] * query(first,
        last));

        k %= sz[rt[i]];
        if (!k) return ret;
        int l = pos[i], r = first + (pos[i] - first + k - 1) %
        sz[rt[i]];
        if (l <= r) return op(ret, query(l, r));
        return op(ret, op(query(l, last), query(first, r)));
    }
}
```

4.11. Dijkstra

Dijkstra (camino mínimo, pesos no negativos)

TIME	SPACE
$O((n+m)\log n)$	$O(n+m)$

Distancias mínimas desde un origen `s` con pesos ≥ 0. `dist[v] = INF` si `v` no es alcanzable. Se ignoran las entradas viejas de la cola con `d > dist[u]`.

Úsese cuando: piden el camino mínimo y todos los pesos son no negativos. Con pesos negativos usar Bellman-Ford o Johnson.

graficas/dijkstra.cpp

```
// Dijkstra: distancias mínimas desde s. Pesos w >= 0. O((n + m) log n).
// g[u] = lista de {v, w}. dist[v] == INF si v es inalcanzable.
// Ojo: con pesos negativos da respuestas incorrectas (usar Bellman-
Ford).
// Úsese cuando: camino mínimo con pesos no negativos, N, M hasta ~2e5.
vector<ll> dijkstra(int s, const vector<vector<pair<int, ll>>>& g) {
    vector<ll> dist(g.size(), INF);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>>
    pq;
    dist[s] = 0;
    pq.push({0, s});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;          // entrada vieja
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) {
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
    }
    return dist;
}
// Ej.: n=3, aristas 0-1 (w=5), 1-2 (w=2), 0-2 (w=9), no dirigido:
//   g[0]={{1,5},{2,9}}, g[1]={{0,5},{2,2}}, g[2]={{0,9},{1,2}}
//   dijkstra(0, g) -> {0, 5, 7}
```

4.12. Floyd-Warshall

TIME	SPACE
$O(n^3)$	$O(n^2)$

Algoritmo que encuentra el camino mínimo entre **todo par de vértices** de un grafo (dirigido o no, con pesos positivos o negativos), y adicionalmente detecta la

existencia de ciclos negativos. Antes de ejecutarlo, la matriz de distancias `d` debe inicializarse con $d[i][i] = 0$ para todo i, y para $i \neq j$, $d[i][j] = w$ si existe una arista (i, j) de peso w, o INF en caso contrario.

Funciones principales:

- `floyd_warshall()`: aplica programación dinámica sobre vértices intermedios `k`, relajando iterativamente $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$ para todo par (i, j). Al finalizar, revisa la diagonal de `d`: si algún $d[i][i] < 0$, significa que existe un ciclo de peso negativo alcanzable desde `i`, y la función retorna `1` (verdadero); en caso contrario retorna `0`.

Es la opción estándar para **all-pairs shortest path** cuando n es pequeño (usualmente $n \leq 400$ a 500 en competencia), evitando así tener que correr Bellman-Ford o Dijkstra desde cada vértice individualmente. Su capacidad de detectar ciclos negativos globales lo hace además útil como verificación rápida de consistencia en sistemas de restricciones de diferencias (**difference constraints**).

graficas/floyd_warshall.cpp

```cpp
// Floyd-Warshall
//
// encontra o menor caminho entre todo
// par de vertices e detecta ciclo negativo
// returna 1 sse ha ciclo negativo
// d[i][i] deve ser 0
// para i != j, d[i][j] deve ser w se ha uma aresta
// (i, j) de peso w, INF caso contrario
//
// O(n^3)

int n;
int d[MAX][MAX];

bool floyd_warshall() {
    for (int k = 0; k < n; k++)
    for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    for (int i = 0; i < n; i++)
        if (d[i][i] < 0) return 1;

    return 0;
}
```

4.13. Johnson (All-Pairs Shortest Path)

TIME	SPACE
$O(nm \log m)$	$O(n^2 + m)$

Algoritmo que resuelve **all-pairs shortest path** con el mismo objetivo que Floyd-Warshall (ver **Floyd-Warshall**), es decir, encuentra el camino mínimo entre todo par de vértices y detecta la existencia de un ciclo negativo, pero con mejor complejidad cuando el grafo es **ralo** (m << n^2): $O(nm \log m)$ en vez de $O(n^3)$. Funciona mediante **reponderación** de aristas (usando potenciales de Bellman-Ford) para eliminar pesos negativos, permitiendo luego correr Dijkstra desde cada vértice.

Funciones principales:

- `johnson(n)`: primero calcula potenciales `h[i]` para cada vértice mediante n iteraciones de relajación estilo Bellman-Ford (equivalente a agregar un vértice ficticio conectado a todos con peso 0); si tras n iteraciones aún es posible relajar alguna arista, existe un ciclo negativo y la función retorna `1`. Luego, para cada vértice `i`, corre Dijkstra sobre el grafo reponderado (donde cada arista (v, u) de peso w pasa a tener peso $w + h[v] - h[u] \geq 0$), almacenando las distancias en `d[i][*]`. Finalmente, revierte la reponderación sumando $h[j] - h[i]$ a cada distancia calculada, obteniendo así las distancias reales en el grafo original.

Es la opción preferida sobre Floyd-Warshall cuando n es grande pero el grafo tiene relativamente pocas aristas (m cercano a n), ya que evita el costo cúbico fijo de Floyd-Warshall a cambio de ejecutar Dijkstra n veces sobre un grafo reponderado sin pesos negativos.

graficas/johnson.cpp

```cpp
// Johnson
//
// funciona igual ao Floyd-Warshall
// encontra o menor caminho entre todo
// par de vertices e retorna 1 sse tem
// ciclo negativo no grafo
//
// O(nm log(m))

vector<pair<int, ll>> g[MAX]; // {vizinho, peso}
ll d[MAX][MAX];
```

```
12
13 bool johnson(int n) {
14     vector<ll> h(n, 0);
15     for (int i = 0; i <= n; i++)
16         for (int v = 0; v < n; v++)
17             for (auto [u, w] : g[v]) if (h[u] > h[v] + w) {
18                 if (i == n) return 1;
19                 h[u] = h[v] + w;
20             }
21
22     for (int i = 0; i < n; i++) {
23         for (int j = 0; j < n; j++) d[i][j] = LINF;
24         d[i][i] = 0;
25         priority_queue<pair<ll, int>> pq;
26         pq.emplace(0, i);
27         while (pq.size()) {
28             auto [ndist, v] = pq.top(); pq.pop();
29             if (-ndist > d[i][v]) continue;
30
31             for (auto [u, w] : g[v]) {
32                 w += h[v] - h[u];
33                 if (d[i][u] > d[i][v] + w) {
34                     d[i][u] = d[i][v] + w;
35                     pq.emplace(-d[i][u], u);
36                 }
37             }
38         }
39         for (int j = 0; j < n; j++)
40             d[i][j] += h[j] - h[i];
41     }
42
43     return 0;
44 }
```

5. Programación Dinámica

Kadane's algorithm

TIME
$O(n)$

Given an array of integers, find the maximum sum subarray.

dp/kadane.cpp

```
4  long long kadane(vector<long long>& arr) {
5      long long max_ending_here = arr[0];
6      long long max_so_far = arr[0];
7      for (size_t i = 1; i < arr.size(); ++i) {
8          max_ending_here = max(arr[i], max_ending_here + arr[i]);
9          max_so_far = max(max_so_far, max_ending_here);
10     }
11     return max_so_far;
12 }
```

Knapsack 0/1

TIME
$O(nW)$

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

dp/knapsack01.cpp

```
4  int knapsack01(vector<int>& w, vector<int>& v, int W) {
5      int n = w.size();
6      vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
7
8
9      for (int i = 1; i <= n; ++i) {
10         for (int j = 0; j <= W; ++j) {
11             dp[i][j] = dp[i-1][j]; // no tomar el objeto i
12             if (w[i-1] <= j)
13                 dp[i][j] = max(dp[i][j], dp[i-1][j - w[i-1]] + v[i-1]);
14         }
15     }
16     return dp[n][W];
17 }
```

Coin Change

dp/coin_change.cpp

```
4  long long coin_change(vector<int>& coins, int X) {
```

```cpp
    vector<long long> dp(X + 1, 0);
    dp[0] = 1; // una forma de formar 0

    for (int c : coins) {
        for (int x = c; x <= X; ++x) {
            dp[x] = (dp[x] + dp[x - c]) % MOD;
        }
    }
    return dp[X];
}
```

5.1. LIS (Longest Increasing Subsequence) con Reconstrucción

TIME	SPACE
$O(n \log n)$	$O(n)$

Calcula y reconstruye explícitamente una **Longest Increasing Subsequence** (LIS) del arreglo `v`, no solo su longitud, usando la técnica clásica de `d[t]` = menor valor final posible entre todas las subsecuencias crecientes de longitud `t+1`. Para permitir subsecuencias **no decrecientes** (en vez de estrictamente crecientes), basta con reemplazar `lower_bound` por `upper_bound`.

Funcionamiento:

- Se mantiene el arreglo `d`, donde `d[t]` almacena el menor valor final entre todas las subsecuencias crecientes de longitud `t` encontradas hasta el momento; `d[0] = -"INF"` actúa como centinela.
- Para cada elemento `v[i]`, se busca mediante `lower_bound` la posición `t` donde debe colocarse dentro de `d` (es decir, la longitud de la subsecuencia creciente más larga que termina en `v[i]`), se actualiza `d[t] = v[i]`, y se guarda en `l[i] = t` la longitud de la LIS que termina exactamente en el índice `i`. Se lleva además el máximo `m` de todas las longitudes vistas.
- Para reconstruir la subsecuencia, se recorre `v` de derecha a izquierda buscando, para cada longitud objetivo `m` (comenzando en la longitud total de la LIS), un índice `p` tal que `l[p] == m`; al encontrarlo se agrega `v[p]` al resultado y se decrementa `m`. Al finalizar se invierte el resultado para obtener el orden correcto.

Es la extensión natural del algoritmo estándar de LIS en $O(n \log n)$ (que solo calcula la longitud), agregando reconstrucción explícita de la subsecuencia sin sacrificar la complejidad asintótica.

dp/lis_recupera.cpp

```cpp
// LIS - recupera
//
// Calcula e retorna uma LIS
//
// O(n.log(n))

template<typename T> vector<T> lis(vector<T>& v) {
    int n = v.size(), m = -1;
    vector<T> d(n+1, INF);
    vector<int> l(n);
    d[0] = -INF;

    for (int i = 0; i < n; i++) {
        // Para non-decreasing use upper_bound()
        int t = lower_bound(d.begin(), d.end(), v[i]) - d.begin();
        d[t] = v[i], l[i] = t, m = max(m, t);
    }

    int p = n;
    vector<T> ret;
    while (p--) if (l[p] == m) {
        ret.push_back(v[p]);
        m--;
    }
    reverse(ret.begin(),ret.end());

    return ret;
}
```

5.2. LIS (Longest Increasing Subsequence) - Tamaño

TIME	SPACE
$O(n \log n)$	$O(n)$

Calcula únicamente la **longitud** de la **Longest Increasing Subsequence** (LIS) de `v`, sin reconstruir la subsecuencia (para eso, ver **LIS con Reconstrucción**). Es la versión más simple y compacta del algoritmo en $O(n \log n)$.

Funcionamiento: se mantiene un arreglo auxiliar **ans**, donde **ans[t]** representa el menor valor final posible entre todas las subsecuencias crecientes de longitud **t+1** encontradas hasta el momento (aunque **ans** en sí **no** es una LIS válida, solo codifica esta información). Para cada elemento **t** de **v**, se busca mediante **lower_bound** la primera posición en **ans** cuyo valor sea $\geq t$: si no existe (**it == ans.end()**), significa que **t** extiende la subsecuencia más larga encontrada hasta ahora, por lo que se agrega al final; si existe, se reemplaza ese valor por **t**, ya que **t** permite continuar subsecuencias futuras con un valor final menor (mejor cota). Al finalizar, el tamaño de **ans** es exactamente la longitud de la LIS. Para permitir subsecuencias **no decrecientes** en vez de estrictamente crecientes, basta con reemplazar **lower_bound** por **upper_bound**.

dp/lis_tamanho.cpp

```cpp
// LIS - tamanho
//
// Calcula o tamanho da LIS
//
// O(n log(n))

template<typename T> int lis(vector<T> &v){
    vector<T> ans;
    for (T t : v){
        // Para non-decreasing use upper_bound()
        auto it = lower_bound(ans.begin(), ans.end(), t);
        if (it == ans.end()) ans.push_back(t);
        else *it = t;
    }
    return ans.size();
}
```

5.3. Divide and Conquer DP (Particionamento en k subarreglos)

TIME	SPACE
$O(kn \log n)$	$O(n)$

Optimización de programación dinámica que particiona un arreglo en exactamente k subarreglos contiguos, minimizando la suma de una función **query(l, r)** (costo de cada subarreglo $[l, r]$) evaluada sobre todas las particiones. Aplica cuando el óptimo satisface la propiedad de **monotonicidad del punto de quiebre** (**opt monotonicity**): si $\text{opt}[i]$ es el mejor punto de corte para el estado **i**, entonces $\text{opt}[i]$ es no decreciente en **i**. Se asume que **query(l, r)** se puede evaluar en $O(1)$ (usualmente mediante prefijos precomputados).

Funciones principales:

- **solve(k, l, r, lk, rk)**: resuelve recursivamente, para la capa **k** de la DP, todos los estados **m** en el rango $[l, r]$, sabiendo de antemano que sus puntos de corte óptimos están acotados dentro de $[\text{lk}, \text{rk}]$. Calcula el punto de corte óptimo **p** para el estado medio **m** mediante búsqueda lineal en $[\max(m, \text{lk}), \text{rk}]$, y usa ese resultado para acotar las llamadas recursivas sobre las mitades izquierda y derecha ($[l, m-1]$ acotado por $[\text{lk}, p]$, y $[m+1, r]$ acotado por $[p, \text{rk}]$), garantizando que el trabajo total por capa sea $O(n \log n)$ en vez de $O(n^2)$.
- **DC(n, k)**: función principal. Inicializa la capa base **dp[n][*] = 0** (costo de no tener más elementos por particionar) y **dp[i][0] = "LINF"** para los demás estados, luego ejecuta **solve** una vez por cada una de las k capas, alternando el arreglo **dp** mediante paridad (**k&1**) para ahorrar memoria (solo se guardan dos capas simultáneamente en vez de $O(kn)$). Retorna el costo mínimo total **dp[0][k&1]**.

Es la técnica estándar para acelerar DPs de partición cuando la función de costo satisface **quadrangle inequality** o una condición de monotonicidad similar, reduciendo la complejidad de $O(kn^2)$ (DP ingenua) a $O(kn \log n)$.

dp/dc_dp.cpp

```cpp
// Divide and Conquer DP
//
// Particiona o array em k subarrays
// minimizando o somatorio das queries
//
// O(k n log n), assumindo quer query(l, r) eh O(1)

ll dp[MAX][2];

void solve(int k, int l, int r, int lk, int rk) {
    if (l > r) return;
    int m = (l+r)/2, p = -1;
    auto& ans = dp[m][k&1] = LINF;
    for (int i = max(m, lk); i <= rk; i++) {
        ll at = dp[i+1][~k&1] + query(m, i);
        if (at < ans) ans = at, p = i;
    }
    solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
```

```
}

ll DC(int n, int k) {
    dp[n][0] = dp[n][1] = 0;
    for (int i = 0; i < n; i++) dp[i][0] = LINF;
    for (int i = 1; i <= k; i++) solve(i, 0, n-i, 0, n-i);
    return dp[0][k&1];
}
```

5.4. Longest Common Subsequence con Reconstrucción (Hirschberg)

TIME	SPACE
$O(nm)$	$O(n+m)$

Calcula la **Longest Common Subsequence** (LCS) entre dos arreglos `s` y `t`, reconstruyendo explícitamente la subsecuencia mediante el algoritmo de Hirschberg, el cual logra recuperar la solución completa usando solo $O(n+m)$ de memoria en vez de los $O(nm)$ que requeriría guardar toda la tabla de DP clásica para **backtracking**. Se basa en **divide and conquer** sobre una de las dos secuencias, combinado con DP calculada desde ambos extremos.

Funciones principales:

- `dp_top(li, ri, lj, rj)`: calcula, usando solo $O(\text{rj} - \text{lj})$ de memoria (una fila `dp[0]`), el valor de la LCS entre `s[li..ri]` y cada prefijo `t[lj..j]` para `j` en [lj, rj], mediante la recurrencia estándar de LCS pero colapsada a una sola fila reutilizada.
- `dp_bottom(li, ri, lj, rj)`: análogo a `dp_top` pero calculado desde el extremo opuesto, obteniendo el valor de la LCS entre `s[li..ri]` y cada sufijo `t[j..rj]`, almacenado en `dp[1]`.
- `solve(ans, li, ri, lj, rj)`: función recursiva principal. En los casos base (`li == ri` o `lj == rj`, es decir, uno de los dos segmentos tiene un solo elemento), busca directamente si ese elemento aparece en el otro segmento y lo agrega a la respuesta. En el caso general, divide `s[li..ri]` por la mitad (`mi`), calcula `dp_top` sobre la mitad izquierda y `dp_bottom` sobre la mitad derecha respecto a todo el rango `t[lj..rj]`, y busca el punto de corte `j_` en `t` que maximiza la suma `dp[0][j] + dp[1][j+1]` (es decir, el mejor punto para dividir `t` de forma consistente con la división de `s`). Luego recursa sobre ambas mitades: `s[li..mi]` con `t[lj..j_]`, y `s[mi+1..ri]` con `t[j_+1..rj]`.
- `lcs(s, t)`: función de entrada; copia los arreglos a los buffers globales `lcs_s`/`lcs_t` y llama a `solve` sobre el rango completo, retornando la subsecuencia común más larga como vector.

Esta técnica es la extensión natural del algoritmo clásico de LCS en $O(nm)$ tiempo, pero optimizando memoria de $O(nm)$ a $O(n+m)$ sin sacrificar la capacidad de reconstruir explícitamente la subsecuencia — crucial cuando n y m son grandes y no cabe la tabla completa en memoria.

dp/lcs_hirschberg.cpp

```cpp
// Longest Common Subsequence
//
// Computa a LCS entre dois arrays usando
// o algoritmo de Hirschberg para recuperar
//
// O(n*m), O(n+m) de memoria

int lcs_s[MAX], lcs_t[MAX];
int dp[2][MAX];

// dp[0][j] = max lcs(s[li...ri], t[lj, lj+j])
void dp_top(int li, int ri, int lj, int rj) {
    memset(dp[0], 0, (rj-lj+1)*sizeof(dp[0][0]));
    for (int i = li; i <= ri; i++) {
        for (int j = rj; j >= lj; j--)
            dp[0][j - lj] = max(dp[0][j - lj],
            (lcs_s[i] == lcs_t[j]) + (j > lj ? dp[0][j-1 - lj] : 0));
        for (int j = lj+1; j <= rj; j++)
            dp[0][j - lj] = max(dp[0][j - lj], dp[0][j-1 -lj]);
    }
}

// dp[1][j] = max lcs(s[li...ri], t[lj+j, rj])
void dp_bottom(int li, int ri, int lj, int rj) {
    memset(dp[1], 0, (rj-lj+1)*sizeof(dp[1][0]));
    for (int i = ri; i >= li; i--) {
        for (int j = lj; j <= rj; j++)
            dp[1][j - lj] = max(dp[1][j - lj],
            (lcs_s[i] == lcs_t[j]) +  (j < rj ? dp[1][j+1 - lj] : 0));
        for (int j = rj-1; j >= lj; j--)
            dp[1][j - lj] = max(dp[1][j - lj], dp[1][j+1 - lj]);
```

```
    }
}

void solve(vector<int>& ans, int li, int ri, int lj, int rj) {
    if (li == ri){
        for (int j = lj; j <= rj; j++)
            if (lcs_s[li] == lcs_t[j]){
                ans.push_back(lcs_t[j]);
                break;
            }
        return;
    }
    if (lj == rj){
        for (int i = li; i <= ri; i++){
            if (lcs_s[i] == lcs_t[lj]){
                ans.push_back(lcs_s[i]);
                break;
            }
        }
        return;
    }
    int mi = (li+ri)/2;
    dp_top(li, mi, lj, rj), dp_bottom(mi+1, ri, lj, rj);

    int j_ = 0, mx = -1;

    for (int j = lj-1; j <= rj; j++) {
        int val = 0;
        if (j >= lj) val += dp[0][j - lj];
        if (j < rj) val += dp[1][j+1 - lj];

        if (val >= mx) mx = val, j_ = j;
    }
    if (mx == -1) return;
    solve(ans, li, mi, lj, j_), solve(ans, mi+1, ri, j_+1, rj);
}

vector<int> lcs(const vector<int>& s, const vector<int>& t) {
    for (int i = 0; i < s.size(); i++) lcs_s[i] = s[i];
    for (int i = 0; i < t.size(); i++) lcs_t[i] = t[i];
    vector<int> ans;
    solve(ans, 0, s.size()-1, 0, t.size()-1);
    return ans;
}
```

5.5. Knapsack (Mochila) con Reconstrucción y Memoria Optimizada

TIME	SPACE
$O(n \cdot \text{cap})$	$O(n + \text{cap})$

Resuelve el problema de la mochila 0/1 (**0/1 Knapsack**) reconstruyendo explícitamente el conjunto de ítems elegidos, usando solo $O(n + \text{cap})$ de memoria en lugar de los $O(n \cdot \text{cap})$ que requeriría la tabla completa de DP para **backtracking** tradicional. Aplica la misma idea de **divide and conquer** que el algoritmo de Hirschberg para LCS (ver **LCS con Reconstrucción**): dividir el conjunto de ítems en dos mitades, calcular la DP desde ambos extremos con memoria colapsada, y combinar los resultados en el punto óptimo.

Funciones principales:

- **`get_dp(x, l, r, cap)`**: calcula, usando solo una fila **`dp[x]`** de tamaño $\text{cap} + 1$, el valor máximo obtenible usando únicamente los ítems en el rango $[l, r]$ para cada capacidad posible de 0 a **`cap`**, mediante la recurrencia estándar de mochila 0/1 (iterando la capacidad en reversa para no reusar un mismo ítem más de una vez).
- **`solve(ans, l, r, cap)`**: función recursiva principal. En el caso base ($l == r$, un solo ítem), agrega el ítem a la respuesta si su peso cabe en la capacidad restante. En el caso general, divide el rango de ítems por la mitad (**`m`**), calcula **`get_dp`** para la mitad izquierda **`[l, m]`** y la derecha **`[m+1, r]`** de forma independiente, y busca la repartición óptima de capacidad **`left_cap`** entre ambas mitades que maximice **`dp[0][j] + dp[1][cap - j]`**. Luego recursa sobre ambas mitades, cada una con su porción de capacidad asignada (**`left_cap`** para la izquierda, **`cap - left_cap`** para la derecha).
- **`knapsack(n, cap)`**: función de entrada; llama a **`solve`** sobre el rango completo de ítems $[0, n-1]$ y la capacidad total **`cap`**, retornando la lista de índices de los ítems seleccionados en la solución óptima.

Es la versión de mochila 0/1 análoga a Hirschberg para LCS: mismo truco de **divide and conquer** con DP bidireccional para lograr reconstrucción de la solución sin pagar el costo de memoria de guardar toda la tabla de **backtracking**.

dp/knapsack_hirschberg.cpp

```
// Mochila
//
```

```cpp
// Resolve mochila, recuperando a resposta
//
// O(n * cap), O(n + cap) de memoria

int v[MAX], w[MAX]; // valor e peso
int dp[2][MAX_CAP];

// DP usando os itens [l, r], com capacidade = cap
void get_dp(int x, int l, int r, int cap) {
    memset(dp[x], 0, (cap+1)*sizeof(dp[x][0]));
    for (int i = l; i <= r; i++) for (int j = cap; j >= 0; j--)
        if (j - w[i] >= 0) dp[x][j] = max(dp[x][j], v[i] + dp[x][j - w[i]]);
}

void solve(vector<int>& ans, int l, int r, int cap) {
    if (l == r) {
        if (w[l] <= cap) ans.push_back(l);
        return;
    }
    int m = (l+r)/2;
    get_dp(0, l, m, cap), get_dp(1, m+1, r, cap);
    int left_cap = -1, opt = -INF;
    for (int j = 0; j <= cap; j++)
        if (int at = dp[0][j] + dp[1][cap - j]; at > opt)
            opt = at, left_cap = j;
    solve(ans, l, m, left_cap), solve(ans, m+1, r, cap - left_cap);
}

vector<int> knapsack(int n, int cap) {
    vector<int> ans;
    solve(ans, 0, n-1, cap);
    return ans;
}
```

SOS DP (Sum over Subsets / Supersets)

TIME	SPACE
$O(n \cdot 2^n)$	$O(2^n)$

Técnica de programación dinámica sobre **bitmasks** que calcula, para cada máscara, la suma de f sobre todos sus subconjuntos (o superconjuntos) en $O(n \cdot 2^n)$, en lugar de $O(3^n)$ que resultaría de iterar explícitamente todos los pares (máscara, submáscara). Se procesa un bit a la vez, propagando la contribución entre máscaras que difieren únicamente en ese bit.

Funciones principales:

- **`sos_dp(f)`** (suma de subconjuntos): dado un arreglo **`f`** indexado por máscara (de tamaño 2^N), retorna un nuevo arreglo donde la posición **`mask`** contiene la suma de $f[\text{submask}]$ sobre todos los **`submask`** $\subseteq$ **`mask`**. Para cada bit **`i`**, recorre todas las máscaras que tienen el bit **`i`** encendido y les suma la contribución de la misma máscara con ese bit apagado.
- **`sos_dp(f)`** (suma de superconjuntos): variante simétrica que retorna, en la posición **`mask`**, la suma de $f[\text{supmask}]$ sobre todos los **`supmask`** $\supseteq$ **`mask`**. Para cada bit **`i`**, recorre las máscaras que tienen el bit **`i`** **apagado** y les suma la contribución de la misma máscara con ese bit encendido.

En ambos casos, **`N = __builtin_ctz(f.size())`** obtiene $\log_2$ del tamaño del arreglo (que debe ser una potencia de 2 exacta, verificado con **`assert`**). Es una herramienta fundamental en problemas de conteo o agregación sobre subconjuntos/superconjuntos de un universo pequeño (usualmente $N \leq$ 20-22 en competencia), como en problemas de **bitmask DP**, convoluciones AND/OR (**Zeta/Möbius transform**), o conteo de máscaras compatibles.

dp/sos_dp.cpp

```cpp
// SOS DP [nohash]
//
// O(n 2^n)

// soma de sub-conjunto
vector<ll> sos_dp(vector<ll> f) {
    int N = __builtin_ctz(f.size());
    assert((1<<N) == f.size());

    for (int i = 0; i < N; i++) for (int mask = 0; mask < (1<<N); mask++)
        if (mask>>i&1) f[mask] += f[mask^(1<<i)];
    return f;
}

// soma de super-conjunto
vector<ll> sos_dp(vector<ll> f) {
    int N = __builtin_ctz(f.size());
    assert((1<<N) == f.size());
```

```
19
20      for (int i = 0; i < N; i++) for (int mask = 0; mask < (1<<N);
        mask++)
21          if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
22      return f;
23  }
```

Subset Sum Acotado (Meet-in-the-Middle Implícito)

TIME	SPACE
$O(n \cdot \max(w))$	$O(\max(w))$

Dado un arreglo de pesos `w` y un umbral `t`, retorna el mayor valor $x \leq t$ tal que existe un subconjunto de `w` cuya suma sea exactamente `x`. Logra una complejidad de $O(n \cdot \max(w))$ tiempo y $O(\max(w))$ memoria, mejorando la DP clásica de subset sum en $O(n \cdot t)$ cuando t es mucho mayor que $\max(w)$ (situación común cuando $t \approx \sum w_i$).

Funcionamiento:

- Primero se consumen "gratis" (sin usar DP) todos los elementos iniciales de `w` mientras la suma prefijo `pref` no exceda `t`, avanzando el índice `k`. Si se logra consumir todo el arreglo sin exceder `t`, la respuesta es directamente `pref`.
- En caso contrario, se define $W = \max(w)$ y se trabaja con una DP relativa (`dp`, de tamaño $2W$), donde `dp[x]` almacena el mayor índice `j` tal que, usando los elementos `w[j..i]` (los "más recientes" procesados hasta el elemento `i`), es posible alcanzar una suma relativa `x` respecto al objetivo desplazado. Esta representación relativa es lo que permite acotar el tamaño de la DP a $O(W)$ en vez de $O(t)$.
- Por cada nuevo elemento `w[i]`, se actualiza `dp` en dos fases: primero una fase de "inclusión hacia adelante" (agregar `w[i]` a estados existentes), y luego una fase de "poda hacia atrás" que descarta del extremo derecho los elementos más antiguos que ya no son necesarios, manteniendo la ventana de $2W$ acotada y consistente.
- Al finalizar, se busca decrementando desde `ans = t` el mayor valor para el cual `dp[W - (t - ans)] >= 0`, es decir, el mayor valor alcanzable que no supere `t`.

Es una técnica avanzada y poco común, útil en problemas donde t es grande pero los pesos individuales están acotados por un W mucho menor, evitando así una DP ingenua de $O(n \cdot t)$ que sería demasiado lenta o consumiría demasiada memoria.

dp/subset_sum_bounded.cpp

```
1   // Subset sum
2   //
3   // Retorna max(x <= t tal que existe subset de w que soma x)
4   //
5   // O(n * max(w))
6   // O(max(w)) de memoria
7
8   int subset_sum(vector<int> w, int t) {
9       int pref = 0, k = 0;
10      while (k < w.size() and pref + w[k] <= t) pref += w[k++];
11      if (k == w.size()) return pref;
12      int W = *max_element(w.begin(), w.end());
13      vector<int> last, dp(2*W, -1);
14      dp[W - (t-pref)] = k;
15      for (int i = k; i < w.size(); i++) {
16          last = dp;
17          for (int x = 0; x < W; x++) dp[x+w[i]] = max(dp[x+w[i]],
            last[x]);
18          for (int x = 2*W - 1; x > W; x--)
19              for (int j = max(0, last[x]); j < dp[x]; j++)
20                  dp[x-w[j]] = max(dp[x-w[j]], j);
21      }
22      int ans = t;
23      while (dp[W - (t-ans)] < 0) ans--;
24      return ans;
25  }
```

6. Teoría de números

6.1. Cribas

Criba de primos con optimizaciones básicas

TIME	SPACE
$O(n \log \log n)$	$O(n)$

Find all the prime numbers in $[1, n]$. This code uses the next optimizations:

- Sieving till root
- Sieving by the odd numbers only

teoria_de_numeros/sieve_with_basic_optimizations.cpp

```
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   vector<bool> sieve(int n) {
5       vector<bool> is_prime(n + 1, true);
```

```
    is_prime[0] = is_prime[1] = false;
    for (int i = 4; i <= n; i += 2) is_prime[i] = false;
    for (int i = 3; i * i <= n; i += 2) {
        if (is_prime[i]) {
            for (int j = i * i; j <= n; j += 2 * i)
                is_prime[j] = false;
        }
    }
    return is_prime;
}
```

Criba de divisores

Produce una criba para obtener la suma de todos los divisores de cada número en $[1, n]$ y otra para dichos divisores.

teoria_de_numeros/criba_divisores.cpp

```
vector<ll> divsSum;
vector<vector<int>> divs;

void divisorsSieve(int n) {
    divsSum.resize(n + 1, 0);
    divs.resize(n + 1);
    for (int i = 1; i <= n; ++i) {
        for (int j = i; j <= n; j += i) {
            divsSum[j] += i;
            divs[j].push_back(i);
        }
    }
}
```

Criba del primo más pequeño

teoria_de_numeros/criba_primo_mas_pequenio.cpp

```
vector<int> lowestPrimeSieve(int n) {
    vector<int> lp(n + 1);
    iota(lp.begin(), lp.end(), 0);
    for (int i = 4; i <= n; i += 2)
        lp[i] = 2;
    for (int i = 3; i * i <= n; i += 2)
        if (lp[i] == i)
            for (int j = i * i; j <= n; j += 2 * i)
                lp[j] = min(lp[j], i);
    return lp;
}
```

Criba del primo más grande

teoria_de_numeros/criba_primo_mas_grande.cpp

```
vector<int> greatestPrimeSieve(int n)
{
    vector<int> gp(n + 1);
    iota(gp.begin(), gp.end(), 0);
    for (int i = 2; i <= n; i++)
        if (gp[i] == i)
            for (int j = 2 * i; j <= n; j += i)
                gp[j] = i;
    return gp;
}
```

6.2. Miller-Rabin (Test de Primalidad Determinístico)

TIME	SPACE
$O(\log n)$	$O(1)$

Test de primalidad **determinístico** (no probabilístico) para $n \leq 3 \cdot 10^{18}$, basado en el test de Miller-Rabin con un conjunto fijo de bases que garantiza corrección exacta en ese rango — a diferencia del Miller-Rabin probabilístico clásico, aquí no hay margen de error. Con las bases usadas, el test funciona garantizado para $n \leq 2^{64}$, y el propio autor señala que funciona para $n \leq 3 \cdot 10^{24}$ si se agregan las bases hasta 41.

Funciones principales:

- `mul(a, b, m)`: multiplicación modular de `a * b mod m` segura para `m` cercano a 2^{63}, evitando **overflow** mediante la técnica de aproximar el cociente con punto flotante de alta precisión (`long double`) y corregir el resultado, en vez de usar `__int128` (más lento en algunas plataformas).
- `pow(x, y, m)`: exponenciación modular rápida ($x^y \bmod m$) mediante **fast power**, construida sobre `mul` para evitar overflow durante las multiplicaciones intermedias.

- `prime(n)`: descarta rápidamente los casos triviales ($n < 2$: falso; $n \leq 3$: primo; `n` par: compuesto), luego descompone $n - 1 = d \cdot 2^r$ (con `d` impar) usando `__builtin_ctzll`. Para cada base `a` del conjunto fijo `{2, 325, 9375, 28178, 450775, 9780504, 1795265022}`, calcula $a^d \bmod n$ y verifica la condición de Miller-Rabin: si el resultado no es 1 ni $n - 1$, eleva al cuadrado repetidamente hasta $r - 1$ veces buscando alcanzar $n - 1$; si nunca se alcanza, `n` es compuesto y se retorna falso. Si todas las bases pasan el test, `n` es primo.

Es la implementación estándar de referencia para verificar primalidad de enteros grandes en competencia (por ejemplo, dentro de factorización tipo Pollard's rho), siendo mucho más rápida que trial division ($O(\sqrt{n})$) para valores grandes de n.

teoria_de_numeros/miller_rabin.cpp

```cpp
// Miller-Rabin
//
// Testa se n eh primo, n <= 3 * 10^18
//
// O(log(n)), considerando multiplicacao
// e exponenciacao constantes

ll mul(ll a, ll b, ll m) {
    ll ret = a*b - ll((long double)1/m*a*b+0.5)*m;
    return ret < 0 ? ret+m : ret;
}

ll pow(ll x, ll y, ll m) {
    if (!y) return 1;
    ll ans = pow(mul(x, x, m), y/2, m);
    return y%2 ? mul(x, ans, m) : ans;
}

bool prime(ll n) {
    if (n < 2) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0) return 0;
    ll r = __builtin_ctzll(n - 1), d = n >> r;

    // com esses primos, o teste funciona garantido para n <= 2^64
    // funciona para n <= 3*10^24 com os primos ate 41
    for (int a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
        ll x = pow(a, d, n);
        if (x == 1 or x == n - 1 or a % n == 0) continue;

        for (int j = 0; j < r - 1; j++) {
            x = mul(x, x, n);
            if (x == n - 1) break;
        }
        if (x != n - 1) return 0;
    }
    return 1;
}
```

6.3. Euclid's Algorithm

The algorithm is based on the formula

$$\gcd(a, b) = \begin{cases} a & b = 0 \\ \gcd(b, a \bmod b) & b \neq 0 \end{cases}$$

6.3.1. Extended Euclid's Algorithm:

Euclid's algorithm can also be extended so that it gives integers x and y for which

$$ax + by = \gcd(a, b)$$

Iterative version

TIME
$O(\log \min(a, b))$

teoria_de_numeros/extended_euclid_algorithm.cpp

```cpp
int gcd(int a, int b, int& x, int& y) {
    x = 1, y = 0;
    int x1 = 0, y1 = 1, a1 = a, b1 = b;
    while (b1) {
        int q = a1 / b1;
        tie(x, x1) = make_tuple(x1, x - q * x1);
        tie(y, y1) = make_tuple(y1, y - q * y1);
        tie(a1, b1) = make_tuple(b1, a1 - q * b1);
    }
    return a1;
}
```

6.4. Euler's Theorem

Euler's totient function $\varphi(n)$ gives the number of integers between $1, \ldots, n$ that are coprime to n.

Any value of $\varphi(n)$ can be calculated from the prime factorization of n using the formula

$$\varphi(n) = \prod_{i=1}^{k} p_i^{\alpha_i - 1}(p_i - 1)$$

> **Teorema 6.4.1 (Euler's theorem)**
>
> For all positive coprime integers x and m
>
> $$x^{\varphi(m)} \bmod m = 1$$

6.5. Solving Equations

We can efficiently solve a Diophantine equation by using the extended Euclid's algorithm which gives integers x and y that satisfy the equation

$$ax + by = \gcd(a, b)$$

A Diophantine equation can be solved exactly when c is divisible by $\gcd(a, b)$.

If a pair (x, y) is a solution, then also all pairs

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right)$$

are solutions, where k is any integer.

7. Combinatoria

7.1. Binomial Coefficients

> **Nota**
>
> For $n < k$ the value of $\binom{n}{k}$ is assumed to be zero.

Basic Improved Implementation

```
int C(int n, int k) {
    double res = 1;
    for (int i = 1; i <= k; ++i)
        res = res * (n - k + i) / i;
    return (int)(res + 0.01);
}
```

Table of binomial coefficients using Pascal's Triangle identity

```
const int maxn = ...;
int C[maxn + 1][maxn + 1];
C[0][0] = 1;
for (int n = 1; n <= maxn; ++n) {
    C[n][0] = C[n][n] = 1;
    for (int k = 1; k < n; ++k)
        C[n][k] = C[n - 1][k - 1] + C[n - 1][k];
}
```

8. Strings

8.1. Trie

Estructura

strings/trie.cpp

```
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);
```

Agregar una cadena

strings/trie.cpp

```
void add_string(string const& s) {
    int v = 0;
```

```
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}
```

8.2. Trie UFMG

TIME	SPACE
insert : $O(\|s\| \cdot \sigma)$ erase, find, count_pref : $O(\|s\|)$	$O(\|s\| \cdot \sigma)$

Estructura de árbol de prefijos (**Trie**) genérica, parametrizable para cualquier alfabeto: `trie T()` construye una trie para el alfabeto de letras minúsculas por defecto, mientras que `trie T(sigma, norm)` permite especificar el tamaño del alfabeto y el carácter mínimo (`norm`), útil para alfabetos como dígitos, mayúsculas, o binarios.

Funciones principales:

- `insert(s)`: inserta la cadena `s` en la trie en $O(|s| \cdot \sigma)$, creando nuevos nodos según sea necesario y llevando en `pref[x]` la cantidad de cadenas insertadas que pasan por el nodo `x` (incluyendo la raíz, que cuenta el total de inserciones). Marca `end[x]` en el nodo final para indicar que ahí termina una cadena completa (permitiendo múltiples inserciones de la misma cadena).
- `erase(s)`: elimina una ocurrencia de `s` de la trie en $O(|s|)$, decrementando `pref` a lo largo del camino y eliminando la arista (`nxt = 0`) cuando un nodo deja de tener cadenas pasando por él, liberando efectivamente esa rama.
- `find(s)`: retorna el índice del nodo correspondiente al final de `s` en la trie, o -1 si `s` no existe como camino completo desde la raíz. Complejidad $O(|s|)$.
- `count_pref(s)`: retorna la cantidad de cadenas almacenadas en la trie que tienen a `s` como prefijo, aprovechando el contador `pref` ya mantenido en cada nodo. Complejidad $O(|s|)$.

Es la base para estructuras más avanzadas como **Aho-Corasick**, **XOR Trie** (para consultas de máximo XOR), o problemas de conteo de prefijos/sufijos comunes entre conjuntos de cadenas.

strings/trie_ufmg.cpp

```cpp
// Trie
//
// trie T() constroi uma trie para o alfabeto das letras minusculas
// trie T(tamanho do alfabeto, menor caracter) tambem pode ser usado
//
// T.insert(s) - O(|s|*sigma)
// T.erase(s) - O(|s|)
// T.find(s) retorna a posicao, -1 se nao achar - O(|s|)
// T.count_pref(s) numero de strings que possuem s como prefixo - O(|s|)

struct trie {
    vector<vector<int>> to;
    vector<int> end, pref;
    int sigma; char norm;
    trie(int sigma_=26, char norm_='a') : sigma(sigma_), norm(norm_) {
        to = {vector<int>(sigma)};
        end = {0}, pref = {0};
    }
    void insert(string s) {
        int x = 0;
        for (auto c : s) {
            int &nxt = to[x][c-norm];
            if (!nxt) {
                nxt = to.size();
                to.push_back(vector<int>(sigma));
                end.push_back(0), pref.push_back(0);
            }
            x = nxt, pref[x]++;
        }
        end[x]++, pref[0]++;
    }
    void erase(string s) {
        int x = 0;
        for (char c : s) {
            int &nxt = to[x][c-norm];
            x = nxt, pref[x]--;
            if (!pref[x]) nxt = 0;
        }
        end[x]--, pref[0]--;
```

```
    }
    int find(string s) {
        int x = 0;
        for (auto c : s) {
            x = to[x][c-norm];
            if (!x) return -1;
        }
        return x;
    }
    int count_pref(string s) {
        int id = find(s);
        return id >= 0 ? pref[id] : 0;
    }
};
```

8.3. String hashing

hash-doble

strings/Hash_Doble.cpp

```
struct Hash {//doble hashing
    vector<ll> h1, h2;
    vector<ll> p1, p2;
    ll p=31;//tomamos esta p para evitar colisiones
    Hash(string &s) {
        int n = s.size();
        h1.resize(n+1, 0);
        h2.resize(n+1, 0);
        p1.resize(n+1, 1);
        p2.resize(n+1, 1);
//hacemos doble hasheo para evitar colisiones
        for (int i = 0; i < n; i++) {
            p1[i+1] = (p1[i] * p) % MOD;//a uno le aplicamos modulo
            p2[i+1] = (p2[i] * p);//el otro ocupamos el modulo
            implicito de ll
//uno con mod 10e9+7 y otro con 2e64 que es LLONG_MAX
            int val = s[i] - 'a' + 1;//hasheamos con el asci

            h1[i+1] = (h1[i] * p + val) % MOD;
            h2[i+1] = (h2[i] * p + val);
        }
    }

    pair<ll,ll> get(ll l, ll r) {//comparamos los 2 hashing
        ll x1 = (h1[r+1] - h1[l] * p1[r-l+1] % MOD + MOD) % MOD;
        ll x2 = (h2[r+1] - h2[l] * p2[r-l+1]);
        return {x1, x2};
    }
};
Hash hs(s);//inicializar
hs.get(i,j);//lugar de sonde se compara
if(hs.get(l,m)==hs.get(i,j))//si 2 cadenas son iguales
```

8.4. KMP — Knuth-Morris-Pratt

KMP — Knuth-Morris-Pratt

TIME	SPACE
$O(n+m)$	$O(m)$

Cuenta ocurrencias de pat en txt. buildPhi construye el arreglo de fallos: phi[j] = prefijo propio más largo de pat[0..j] que es sufijo. Al fallar retrocede a phi[i-1] en lugar de reiniciar.

strings/kmp.cpp

```
vll buildPhi(const vll& pat) {
    ll m = pat.size();
    vll phi(m, 0);
    for (ll j = 1, i = 0; j < m; j++) {
        while (i > 0 && pat[i] != pat[j]) i = phi[i - 1];
        if (pat[i] == pat[j]) i++;
        phi[j] = i;
    }
    return phi;
}

ll KMP(const vll& pat, const vll& txt) {
    ll m = pat.size(), n = txt.size();
    if (m == 0) return 0;
    vll phi = buildPhi(pat);
    ll matches = 0;
    for (ll i = 0, j = 0; j < n; j++) {
```

```
        while (i > 0 && pat[i] != txt[j]) i = phi[i - 1];
        if (pat[i] == txt[j]) i++;
        if (i == m) { matches++; i = phi[i - 1]; }
    }
    return matches;
}
```

Bordes de un string

TIME	SPACE
$O(n)$	$O(n)$

Dado phi de un string, devuelve todas las longitudes de bordes no triviales (prefijo = sufijo, distinto del string completo), de mayor a menor. Se obtiene siguiendo la cadena phi[n-1], phi[phi[n-1]-1], ... hasta llegar a 0.

strings/Borders_KMP.cpp

```
vll getBorders(const vll& phi) {
    ll n = phi.size();
    vll borders;
    for (ll i = phi[n - 1]; i; i = phi[i - 1])
        borders.push_back(i);
    return borders;
}
```

Conteo de ocurrencias de cada prefijo

TIME	SPACE
$O(n)$	$O(n)$

Dado phi de un string, cnt[L] = cuántas veces aparece el prefijo de longitud L como substring dentro de todo el string (incluyendo su propia ocurrencia como prefijo). Se calcula acumulando conteos por valor de phi y propagándolos hacia abajo en la cadena de bordes.

strings/Occurrences_KMP.cpp

```
vll countPrefixOccurrences(const vll& phi) {
    ll n = phi.size();
    vll cnt(n + 1, 0);
    for (ll i = 0; i < n; i++)
        cnt[phi[i]]++;
    for (ll i = n - 1; i > 0; i--)
        cnt[phi[i - 1]] += cnt[i];
    for (ll i = 0; i <= n; i++)
        cnt[i]++;
    return cnt;
}
```

9. Manipulación de bits

9.1. Máscara de bits

9.1.1. Manipulación individual de bits

Establece en 1 el k-ésimo bit de `x`:

```
#define ON(x, k) ((x) | (1LL << (k)))
```

Establece en 0 el k-ésimo bit de `x`:

```
#define OFF(x, k) ((x) & ~(1LL << (k)))
```

Invierte el k-ésimo bit de `x`:

```
#define TOGGLE(x, k) ((x) ^ (1LL << (k)))
```

Comprueba si el k-ésimo bit de `x` es 1:

```
#define CHECK(x, k) ((x) & (1LL << (k)))
```

9.1.2. Working with the rightmost 1-bit

`x & -x`: Isolates the rightmost set bit (keeps only the least significant bit that is 1, sets all others to 0).

`x & (x - 1)`: Removes the rightmost set bit from `x` (turns the rightmost 1 to 0).

`x | (x - 1)`: Sets all trailing zeros of `x` to 1.

9.1.3. Creating useful bit masks

`(1 << n) - 1`: Creates a bit mask with the first n bits set to 1.

9.1.4. Common bit manipulation tricks

- A positive number `x` is a power of two exactly when `x & (x - 1) = 0`.

`x ^ A ^ B`: Determines which is not the current value of `x` between `A` and `B`.

9.1.5. Bit operations as math alternatives

`x << k`: equivalent to `x * pow(2, k)`

`x >> k`: equivalent to `x / pow(2, k)` (integer division)
`x & ((1 << k) - 1)`: equivalent to `x % pow(2, k)`
`A + B`: equivalent to `(A ^ B) + 2 * (A & B)` or `(A | B) + (A & B)`

9.1.6. GNU C++ compiler built-in functions

`__builtin_clz(x)`: the number of leading zeros (zeros on the left side of the bit representation).
`__builtin_ctz(x)`: the number of trailing zeros (zeros on the right side of the bit representation).
`__builtin_popcount(x)`: the number of ones in the bit representation.
`__builtin_parity(x)`: the parity (even or odd) of the number of ones in the bit representation.

Example of use

```
int x = 5328; // 00000000000000000001010011010000
cout << __builtin_clz(x) << "\n";     // 19
cout << __builtin_ctz(x) << "\n";     // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n";   // 1
```

> **Nota**
>
> The above functions only support `int` numbers, but there are also `long long` versions of the functions available with the suffix `ll` (e.g. `__builtin_popcountll(x)`).

9.2. Representing Sets

Every subset of a set $\{0, 1, 2, ..., n-1\}$ can be represented as an n bit integer whose one bits indicate which elements belong to the subset.

Goes through the subsets with exactly k elements

```
for (int b = 0; b < (1<<n); b++) {
    if (__builtin_popcount(b) == k) {
        // process subset b
    }
}
```

Goes through the subsets of a set x

```
int b = 0;
do {
    // process subset b
} while (b=(b-x)&x);
```

The idea is that the formula $b - x$ detects the rightmost one bit in x that is zero in b. This bit becomes one and all bits after it become zero. Then the and operation ensures that the resulting value is a subset of x. Note that $b - x$ equals $-(x - b)$ so we can think that we first remove all one bits that appear in b and then invert the value and add one.

10. Matemáticas

10.1. Operaciones básicas con matrices

Operaciones básicas con matrices

matematicas/matrix_exponentiation.cpp

```
#include<bits/stdc++.h>
using namespace std;

const int mod = 998244353;

struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
  Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0; a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
      }
```

```
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] %
          mod) % mod;
        }
      }
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) { return *this = (*this) +
  b; }
  inline Mat& operator -= (const Mat& b) { return *this = (*this) -
  b; }
  inline Mat& operator *= (const Mat& b) { return *this = (*this) *
  b; }
  inline bool operator == (const Mat& b) { return a == b.a; }
  inline bool operator != (const Mat& b) { return a != b.a; }
};

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n; long long k; cin >> n >> k;
  Mat a(n, n);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      cin >> a.a[i][j];
    }
  }
  Mat ans = a.pow(k);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      cout << ans.a[i][j] << ' ';
    }
    cout << '\n';
  }
  return 0;
}
// https://judge.yosupo.jp/problem/pow_of_matrix
```

10.2. Division Trick (Floor Division Blocks)

TIME	SPACE
$O(\sqrt{n})$	$O(1)$

Técnica que enumera, en $O(\sqrt{n})$ iteraciones, todos los valores distintos de $\lfloor \frac{n}{i} \rfloor$ para i entre 1 y n, agrupándolos en bloques $[l, r]$ donde $\lfloor \frac{n}{i} \rfloor$ es constante. En cada iteración del ciclo, `l` marca el inicio del bloque actual y `r = n / (n / l)` calcula el mayor índice `i` tal que $\lfloor \frac{n}{i} \rfloor = \lfloor \frac{n}{l} \rfloor$, permitiendo saltar directamente al siguiente bloque en la próxima iteración (`l = r + 1`).

Es una herramienta fundamental para acelerar sumatorias o productos que involucran divisiones enteras, como $\sum_{i=1}^{n} \lfloor \frac{n}{i} \rfloor$ (usada en problemas de conteo de divisores) o convoluciones de funciones multiplicativas, reduciendo el costo de $O(n)$ a $O(\sqrt{n})$ al aprovechar que $\lfloor \frac{n}{i} \rfloor$ toma a lo sumo $O(\sqrt{n})$ valores distintos. Puede

combinarse fácilmente con estructuras de prefijos (por ejemplo, sumas de una función multiplicativa) para responder consultas del tipo $\sum_{i=1}^{n} f(i) \cdot \lfloor \frac{n}{i} \rfloor$ en $O(\sqrt{n})$.

matematicas/division_trick.cpp

```cpp
// Division Trick
//
// Gera o conjunto n/i, pra todo i, em O(sqrt(n))
// copiei do github do tfg50

for(int l = 1, r; l <= n; l = r + 1) {
    r = n / (n / l);
    // n / i has the same value for l <= i <= r
}
```

11. Teoría

11.1. Teoría de números

11.1.1. Algunas funciones relevantes

- La función contadora de primos $\pi(n)$ da el *número de primos hasta* n. Esta función es asintótica a $\frac{n}{\ln(n)}$

x	10	10^2	10^3	10^4	10^5	10^6	10^7	10^8	10^9
$\pi(x)$	4	25	168	1,229	9,592	78,498	664,579	5,761,455	50,847,534

- Número de divisores de un entero n:

$$\tau(n) = \prod_{i=1}^{k} (\alpha_i + 1)$$

  - Un **número altamente compuesto** es un entero positivo que tiene más divisores que cualquier entero positivo menor que él.
  - Un **número altamente compuesto superior** es un número natural que, en un sentido riguroso y específico, posee una gran cantidad de divisores.

  Algunos números altamente compuestos superiores son:

n	6	60	360	5040	55440	720720	4324320	21621600
$\tau(n)$	4	12	24	60	120	240	384	576

n	367567200	6983776800	13967553600	321253732800
$\tau(n)$	1152	2304	2688	5376

  El número $18401055938125660800 \approx 2 \times 10^{18}$ es altamente compuesto y posee 184320 divisores.

  Para números hasta 10^{88} se cumple que $\tau(n) < 3.6\sqrt[3]{n}$.

- Suma de divisores de un entero n:

$$\sigma(n) = \prod_{i=1}^{k} (1 + p_i + \ldots + p_i^{\alpha_i}) = \prod_{i=1}^{k} \frac{p_i^{\alpha_i+1} - 1}{p_i - 1}$$

- Producto de divisores de un entero n:

$$n^{\frac{\tau(n)}{2}}$$

11.1.2. Aritmética modular

Definición 11.1.1 (Modular multiplicative inverse)

The *modular multiplicative inverse* of x with respect to m is a value $\text{inv}_m(x)$ such that

$$x \cdot \text{inv}_m(x) \bmod m = 1$$

If m is prime

$$\text{inv}_m(x) = x^{m-2}$$

- Factorial inverso modular: $\text{inv}_p((x-1)!) \equiv \big(x \cdot \text{inv}_p(x!)\big) \bmod p$

11.2. Combinatoria

11.2.1. Números de Fibonacci

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

11.2.2. Números de Catalán

Fórmula recursiva:

$$C_0 = C_1 = 1$$

$$C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}, \quad n \geq 2$$

Fórmulas analíticas:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!} = \prod_{k=2}^{n}\frac{n+k}{k}, \quad n \geq 0$$

11.2.2.1. Aplicaciones

El número de Catalán C_n es la solución para:

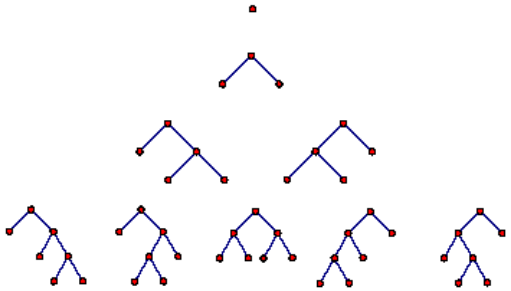
- Número de secuencias de paréntesis correctas formadas por n paréntesis de apertura y n paréntesis de cierre.

  $$((()))\qquad ()(())\qquad ()()()\qquad (())()\qquad \cdots$$

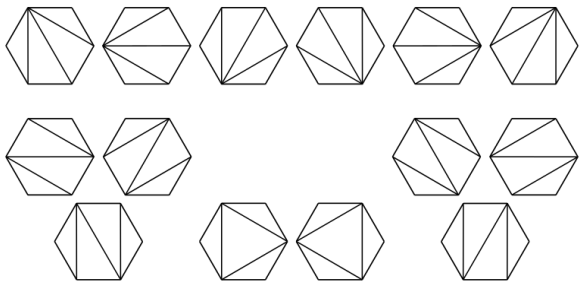
- Las aplicaciones sucesivas de un operador binario pueden representarse mediante un árbol binario completo. Un árbol binario enraizado es completo si cada vértice tiene exactamente dos hijos o ninguno. De esto se sigue que C_n es el número de árboles binarios completos enraizados con $n+1$ hojas (los vértices no están numerados).

  Ejemplo. Para $n+1 = 4$ factores a, b, c, d:

  $$a(b(cd))\quad a((bc)d)\quad (ab)(cd)\quad ((ab)c)d\quad (a(bc))d$$



- El número de triangulaciones de un polígono convexo con $n+2$ lados (es decir, el número de particiones del polígono en triángulos disjuntos usando diagonales).
  Ejemplo. Caso para $n = 4$



- El número de formas de conectar los $2n$ puntos de una circunferencia para formar n cuerdas disjuntas.
- El número de árboles binarios completos no isomorfos con n nodos internos (es decir, nodos que tienen al menos un hijo).
- El número de caminos monótonos en una retícula desde el punto $(0,0)$ hasta el punto (n,n) en una cuadrícula de tamaño $n \times n$, que no pasan por encima de la diagonal principal (i.e. conectando $(0,0)$ a (n,n)).
- El número de permutaciones de longitud n que pueden ordenarse con una pila (stack-sortable permutation). Se puede demostrar que la reordenación es ordenable con una pila si y solo si no existe un índice $i < j < k$ tal que $a_k < a_i < a_j$.
- El número de particiones no cruzadas de un conjunto de n elementos.
- El número de formas de cubrir la escalera $1, \cdots, n$ usando n rectángulos. La escalera consta de n columnas, donde la i-ésima columna tiene altura i.

11.2.3. Identidades binomiales

- Pascal's Triangle:

  $$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

- Symmetry rule:

  $$\binom{n}{k} = \binom{n}{n-k}$$

- Factoring in:

  $$\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$$

- Sum over k:

  $$\sum_{k=0}^{n}\binom{n}{k} = 2^n$$

- Sum over n:

  $$\sum_{m=0}^{n}\binom{m}{k} = \binom{n+1}{k+1}$$

- Sum over n and k:

  $$\sum_{k=0}^{m}\binom{n+k}{k} = \binom{n+m+1}{m}$$

- Sum of the squares:

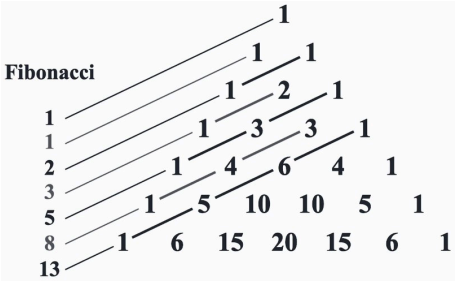
$$\binom{n}{0}^2 + \binom{n}{1}^2 + \cdots + \binom{n}{n}^2 = \binom{2n}{n}$$

- Weighted sum:

$$1\binom{n}{1} + 2\binom{n}{2} + \cdots + n\binom{n}{n} = n2^{n-1}$$

- Connection with the Fibonacci numbers:

$$\binom{n}{0} + \binom{n-1}{1} + \cdots + \binom{n-k}{k} + \cdots + \binom{0}{n} = F_{n+1}$$



11.2.4. Estrellas y barras

Teorema 11.2.1

El número de formas de colocar n objetos idénticos en k cajas etiquetadas es

$$\binom{n+k-1}{n}$$

Utilizando este resultado podemos contar el número de sumas de k enteros acotados inferiormente, i.e, el número de soluciones para la ecuación

$$x_1 + x_2 + \cdots + x_k = n$$

con $x_i \geq a_i$.

La solución es:

$$\binom{n - (a_1 + a_2 + \cdots + a_k) + k - 1}{n}$$

Cuando los x_i están acotados superiormente, podemos usar el principio de inclusión--exclusión para contar el número de soluciones.

11.2.5. Principio de inclusión-exclusión

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{i=1}^{n} |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| + \cdots + (-1)^{n-1} \ |A_1 \cap A_2 \cap \cdots \cap A_n|$$

Forma compacta:

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_{\varnothing \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right|$$

11.2.6. Derangement (subfactorial)

Un desarreglo (derangement) es una permutación que no tiene puntos fijos. Sea d_n el número de desarreglos de una secuencia $1 \ldots n$. Se tiene la recurrencia $d_n = (n-1)(d_{n-1} + d_{n-2})$. Además, d_n es el entero más cercano a $\frac{n!}{e}$.

$$d_n = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!}$$

$$D_n = \sum_{i=0}^{n} (-1)^i \binom{n}{i} (n-i)! \approx \frac{n!}{e}$$

$$D_n = \begin{cases} 0 & \text{si } n = 1 \\ 1 & \text{si } n = 2 \\ (n-1)(D_{n-2} + D_{n-1}) & \text{si } n \geq 3 \end{cases}$$

$$D_n = nD_{n-1} + (-1)^n \quad \text{para} \quad n \geq 1$$

11.3. Primeros términos de algunas sucesiones básicas

n	p_n	F_n	C_n	2^n	$n!$	$!n$
0	—	0	1	1	1	1
1	2	1	1	2	1	0
2	3	1	2	4	2	1
3	5	2	5	8	6	2
4	7	3	14	16	24	9
5	11	5	42	32	120	44
6	13	8	132	64	720	265
7	17	13	429	128	5040	1854
8	19	21	1430	256	40320	14833
9	23	34	4862	512	362880	133496
10	29	55	16796	1024	3628800	1334961
11	31	89	58786	2048	39916800	14684570
12	37	144	208012	4096	479001600	176432560
13	41	233	742900	8192	6227020800	2290792932
14	43	377	2674440	16384	87178291200	32071101049
15	47	610	9694845	32768	1307674368000	481066515734

11.4. Geometría

11.4.1. Ternas pitagóricas

$$a^2 + b^2 = c^2$$

Every Pythagorean triple (a, b, c) is generated uniquely by

$$a = k(m^2 - n^2), b = k(2mn), c = k(m^2 + n^2)$$

where m, n, and k are positive integers with $m > n$, and with m and n coprime and not both odd.

11.4.2. Trigonometría

Sean a, b, c las longitudes de los lados y α, β, γ sus ángulos opuestos, entonces:
Semiperímetro:

$$s = \frac{a+b+c}{2}$$

Área:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

Circunradio:

$$R = \frac{abc}{4A}$$

Inradio:

$$r = \frac{A}{s}$$

Longitud de la mediana (divide el triángulo en dos triángulos de igual área):

$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$$

Longitud de la bisectriz (divide los ángulos en dos):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Ley de senos:

$$\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$$

Ley de cosenos:

$$a^2 = b^2 + c^2 - 2bc\cos\alpha$$

Ley de tangentes:

$$\frac{a+b}{a-b} = \frac{\tan\left(\frac{\alpha+\beta}{2}\right)}{\tan\left(\frac{\alpha-\beta}{2}\right)}$$

11.4.2.1. Suma de ángulos

$$\sin(\alpha \pm \beta) = \sin\alpha\cos\beta \pm \cos\alpha\sin\beta$$
$$\cos(\alpha \pm \beta) = \cos\alpha\cos\beta \mp \sin\alpha\sin\beta$$
$$\tan(\alpha \pm \beta) = \frac{\tan\alpha \pm \tan\beta}{1 \mp \tan\alpha\tan\beta}$$

11.4.2.2. Transformación de suma a producto

$$\sin\alpha \pm \sin\beta = 2\sin\frac{\alpha \pm \beta}{2}\cos\frac{\alpha \mp \beta}{2}$$

$$\cos\alpha + \cos\beta = 2\cos\frac{\alpha + \beta}{2}\cos\frac{\alpha - \beta}{2}$$

$$\cos\alpha - \cos\beta = -2\sin\frac{\alpha + \beta}{2}\sin\frac{\alpha - \beta}{2}$$

$$\tan\alpha \pm \tan\beta = \frac{\sin(\alpha \pm \beta)}{\cos\alpha\cos\beta}$$

11.4.3. Producto punto

$$\vec{a}\cdot\vec{b} = ab\cos(\theta)$$

- El producto punto es positivo si el ángulo entre ellos es agudo.
- Es negativo si el ángulo es obtuso.
- Es cero si son ortogonales (forman un ángulo recto).

11.4.3.1. Propiedades

1. Norma al cuadrado de $\boldsymbol{a}$:

$$|\vec{a}|^2 = \vec{a}\cdot\vec{a}$$

2. Longitud de $\vec{a}$:

$$|\vec{a}| = \sqrt{\vec{a}\cdot\vec{a}}$$

3. Proyección de $\vec{b}$ sobre $\vec{a}$:

$$\text{proj}_a b = \frac{\vec{a}\cdot\vec{b}}{|\vec{b}|^2}\vec{a}$$

4. Ángulo entre vectores:

$$\arccos\left(\frac{\vec{a}\cdot\vec{b}}{|\vec{a}|\ |\vec{b}|}\right)$$

11.5. Sumas y series

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \qquad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} i^3 = \frac{n^2(n+1)^2}{4} = \left(\sum_{i=1}^{n} i\right)^2 \qquad \sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

$$\sum_{k=a}^{b} c^k = \frac{c^{b+1} - c^a}{c-1}, \quad c \neq 1$$

$$g_k(n) = \sum_{i=1}^{n} i^k = \frac{1}{k+1}\left(n^{k+1} + \sum_{j=1}^{k}\binom{k+1}{j+1}(-1)^{j+1} g_{k-j}(n)\right)$$

$$\sum_{i=0}^{n} ic^i = \frac{nc^{n+2} - (n+1)c^{n+1} + c}{(c-1)^2}, \quad c \neq 1$$

$$\sum_{i=0}^{\infty} ic^i = \frac{c}{(1-c)^2}, \quad |c| < 1$$

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots, -\infty < x < \infty$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots, -1 < x \leq 1$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \cdots, -1 \leq x \leq 1$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots, -\infty < x < \infty$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots, -\infty < x < \infty$$

11.6. DP

- **Divide and Conquer Optimization:** Utilizada en problemas del tipo:

$$\text{dp}[i][j] = \min_{k<j}\{\text{dp}[i-1][k] + C[k][j]\}$$

  donde el objetivo es dividir el subsegmento hasta j en i segmentos con algún costo. La optimización es válida si:

$$A[i][j] \leq A[i][j+1]$$

  donde $A[i][j]$ es el valor de k que minimiza la transición.

- **Knuth Optimization:** Aplicable cuando:

$$\text{dp}[i][j] = \min_{i<k<j}\{\text{dp}[i][k] + \text{dp}[k][j]\} + C[i][j]$$

  y se satisface la condición de monotonía:

$$A[i][j-1] \leq A[i][j] \leq A[i+1][j]$$

  con $A[i][j]$ siendo el índice k que minimiza la transición.

- **Slope Trick:** Técnica usada para manejar funciones lineales por partes y convexas. La función se representa mediante puntos donde la derivada cambia, que pueden manipularse con `multiset` o `heap`. Útil para mantener el mínimo de funciones acumuladas en forma de envolventes convexas.
- **Otras Técnicas y Trucos Importantes:**
  - **FFT (Fast Fourier Transform):** Convolución eficiente de vectores.
  - **CHT (Convex Hull Trick):** Optimización para DP con funciones lineales y monotonía.
  - **Aliens Trick:** Técnica para binarizar el costo en problemas de optimización paramétrica (generalmente en problemas con límite en el número de grupos/segmentos).
  - **Bitset:** Utilizado para optimizaciones de espacio y tiempo en DP de subconjuntos o sumas parciales, especialmente en problemas de mochila.

11.7. Geometría

- **Fórmula de Euler:** En un grafo planar o poliedro convexo, tenemos:

$$V - E + F = 2$$

  donde V es el número de vértices, E el número de aristas y F el número de caras.
- **Teorema de Pick:** Para polígonos con vértices en coordenadas enteras:

$$\text{Área} = i + \frac{b}{2} - 1$$

  donde i es el número de puntos interiores y b el número de puntos sobre el perímetro.
- **Teorema de las Dos Orejas (Two Ears Theorem):** Todo polígono simple con más de tres vértices posee al menos dos "orejas" — vértices que pueden eliminarse sin generar intersecciones. La eliminación repetida de las orejas resulta en una triangulación del polígono.
- **Incentro de un Triángulo:** Es el punto de intersección de las bisectrices internas y centro de la circunferencia inscrita. Si a, b y c son las longitudes de los lados opuestos a los vértices $A(X_a, Y_a)$, $B(X_b, Y_b)$ y $C(X_c, Y_c)$, entonces el incentro (X, Y) está dado por:

$$X = \frac{aX_a + bX_b + cX_c}{a+b+c}, \quad Y = \frac{aY_a + bY_b + cY_c}{a+b+c}$$

- **Triangulación de Delaunay:** Una triangulación de un conjunto de puntos en el plano tal que ningún punto está dentro del círculo circunscrito de ningún triángulo. Esta triangulación:
  - Maximiza el ángulo mínimo entre todos los triángulos.
  - Contiene el árbol generador mínimo (MST) euclidiano como subconjunto.
- **Fórmula de Brahmagupta:** Para calcular el área de un cuadrilátero cíclico (todos los vértices sobre una circunferencia), con lados a, b, c y d:

$$s = \frac{a+b+c+d}{2}, \quad \text{Área} = \sqrt{(s-a)(s-b)(s-c)(s-d)}$$

  Si $d = 0$ (es decir, un triángulo), se reduce a la fórmula de Herón:

$$\text{Área} = \sqrt{(s-a)(s-b)(s-c)s}$$

11.8. Grafos

- **Fórmula de Euler (para grafos planares):**

$$V - E + F = 2$$

  donde V es el número de vértices, E el número de aristas y F el número de caras.
- **Lema del Apretón de Manos (Handshaking Lemma):** El número de vértices con grado impar en un grafo es par.
- **Teorema de Kirchhoff (conteo de árboles generadores):** Arme la matriz M tal que:

$$M_{i,i} = \deg(i), \quad M_{i,j} = \begin{cases} -1 \text{ si existe la arista } \ i-j \\ 0 \text{ en caso contrario} \end{cases}$$

  El número de árboles generadores (spanning trees) es el determinante de cualquier cofactor de M (elimine una fila y una columna).
- **Condiciones para Camino Hamiltoniano:**
  - **Teorema de Dirac:** Si todos los vértices tienen grado $\geq \frac{n}{2}$, el grafo contiene un camino Hamiltoniano.
  - **Teorema de Ore:** Si para todo par de vértices no adyacentes u y v se cumple $\deg(u) + \deg(v) \geq n$, entonces el grafo posee un camino Hamiltoniano.
- **Algoritmo de Borůvka:** Mientras el grafo no sea conexo, para cada componente conexa elija la arista de menor costo que sale de ella. Esta técnica construye el árbol generador mínimo (MST).
- **Árboles:**
  - Existen C_n árboles binarios con n vértices (C_n es el n-ésimo número de Catalan).
  - Existen C_{n-1} árboles enraizados con n vértices.
  - **Fórmula de Cayley:** Existen n^{n-2} árboles con vértices etiquetados de 1 a n.
  - **Código de Prüfer:** Elimine iterativamente la hoja con menor etiqueta y agregue la etiqueta del vecino al código, hasta que queden dos vértices.
- **Flujo en Redes:**

- **Corte Mínimo:** Después de ejecutar el algoritmo de flujo máximo, un vértice u está del lado de la fuente si nivel$[u] \neq -1$.
- **Máximo de Caminos Disjuntos:**
  - **Aristas disjuntas:** Use flujo máximo con capacidades iguales a 1 en todas las aristas.
  - **Vértices disjuntos:** Divida cada vértice v en v_{in} y v_{out}, conectados por una arista de capacidad 1. Las aristas que entran van hacia v_{in} y las que salen, salen de v_{out}.
- **Teorema de König:** En un grafo bipartido:

$$\text{Cobertura mínima de vértices} = \text{Matching máximo}$$

  El complemento de la cobertura mínima de vértices es el conjunto independiente máximo.
- **Coberturas:**
  - **Vertex Cover mínimo:** Los vértices de la partición X que **no** están del lado de la fuente en el corte mínimo, y los vértices de la partición Y que **sí** están del lado de la fuente.
  - **Independent Set máximo:** Complemento de la cobertura mínima de vértices.
  - **Edge Cover mínimo:** Es N − matching, tomando las aristas del matching más las aristas restantes necesarias para cubrir los vértices no cubiertos.
- **Path Cover:**
  - **Node-disjoint path cover mínimo:** Duplique los vértices en tipo A y tipo B y cree un grafo bipartido con aristas de $A \to B$. El path cover es N − matching.
  - **General path cover mínimo:** Cree aristas de $A \to B$ siempre que exista un camino de A a B en el grafo. El resultado también es N − matching.
- **Teorema de Dilworth:** El path cover mínimo en un grafo dirigido acíclico es igual a la **antichain máxima** (conjunto de vértices sin caminos entre ellos).
- **Teorema del Matrimonio de Hall:** Un grafo bipartido posee un matching completo del lado X si:

$$\forall W \subseteq X, \quad |W| \leq |\text{vecinos}(W)|$$

- **Flujo Viable con Capacidades Inferiores y Superiores:** Para una red sin fuente ni sumidero:
  - Sustituya la capacidad de cada arista por $c_{\text{upper}} - c_{\text{lower}}$.
  - Cree una nueva fuente S y un sumidero T.
  - Para cada vértice v, calcule:

$$M[v] = \sum_{\text{aristas entrantes}} c_{\text{lower}} - \sum_{\text{aristas salientes}} c_{\text{lower}}$$

  - Si $M[v] > 0$, agregue la arista (S, v) con capacidad $M[v]$; si $M[v] < 0$, agregue (v, T) con capacidad $-M[v]$.
  - Si todas las aristas de S están saturadas en el flujo máximo, entonces existe un flujo viable. El flujo viable final es el flujo calculado más los valores de c_{lower}.

12. Otros

12.1. Conectividad Dinámica Offline (Divide and Conquer + DSU con Rollback)

TIME	SPACE
$O(n \log^2 n)$	$O(n+q)$

Resuelve conectividad dinámica de forma **offline**: dado un conjunto de aristas que se insertan y eliminan a lo largo del tiempo (cada arista activa durante un intervalo de tiempo conocido de antemano), responde consultas sobre el número de componentes conexas en cada instante. Combina **divide and conquer sobre el tiempo** con un DSU con rollback (sin compresión de caminos, para permitir deshacer uniones).

DSU con rollback (`namespace data`):

- `build(n2)`: inicializa el DSU con `n2` elementos, cada uno en su propia componente, y `ans = n` (número inicial de componentes).
- `find(k)`: encuentra la raíz de `k` sin compresión de caminos (necesaria para poder revertir).
- `add(x)`: une los conjuntos de `x.first` y `x.second` por tamaño. Si ya estaban unidos, apila `-1` como marca de "operación nula"; si no, decrementa `ans` y apila la raíz que fue absorbida, para poder deshacerlo luego.
- `query()`: retorna el número actual de componentes conexas (`ans`).
- `rollback()`: deshace la última unión registrada en la pila `S`, restaurando tamaños y punteros, e incrementando `ans` si la unión deshecha no era nula.

Divide and conquer sobre el tiempo (`solve(l, r)`):

- Cada arista `qu[i]` tiene asociado en `ponta[i]` el otro extremo de su intervalo de vida (o -1 si `i` representa una consulta puntual en vez de una arista). Para resolver el rango de tiempo $[l, r]$, `solve` agrega temporalmente (mediante `data::add`) todas las aristas cuyo intervalo de vida cubre completamente uno de los dos sub-rangos $[l, m]$ o $[m+1, r]$ pero no ambos, resuelve recursivamente ese sub-rango con la estructura ya actualizada, y luego revierte (`rollback`) exactamente esas adiciones antes de continuar con el otro sub-rango. Cuando el rango se reduce a un único

instante ($l \geq r$), la estructura `data` refleja el estado correcto en ese instante y se guarda la respuesta en `ans[l]` mediante `data::query()`.

Esta técnica evita mantener una estructura de conectividad completamente dinámica (que requeriría **link-cut trees** u otras estructuras más complejas), a cambio de resolver el problema **offline** con un factor $\log n$ adicional proveniente de la recursión de **divide and conquer**.

otros/dc_conectividade.cpp

```cpp
// Conectividade Dinamica DC
//
// Offline com Divide and Conquer e
// DSU com rollback
// O(n log^2(n))

typedef pair<int, int> T;

namespace data {
    int n, ans;
    int p[MAX], sz[MAX];
    stack<int> S;

    void build(int n2) {
        n = n2;
        for (int i = 0; i < n; i++) p[i] = i, sz[i] = 1;
        ans = n;
    }
    int find(int k) {
        while (p[k] != k) k = p[k];
        return k;
    }
    void add(T x) {
        int a = x.first, b = x.second;
        a = find(a), b = find(b);
        if (a == b) return S.push(-1);
        ans--;
        if (sz[a] > sz[b]) swap(a, b);
        S.push(a);
        sz[b] += sz[a];
        p[a] = b;
    }
    int query() {
        return ans;
    }
    void rollback() {
        int u = S.top(); S.pop();
        if (u == -1) return;
        sz[p[u]] -= sz[u];
        p[u] = u;
        ans++;
    }
};

int ponta[MAX]; // outra ponta do intervalo ou -1 se for query
int ans[MAX], n, q;
T qu[MAX];

void solve(int l = 0, int r = q-1) {
    if (l >= r) {
        ans[l] = data::query(); // agora a estrutura ta certa
        return;
    }
    int m = (l+r)/2, qnt = 1;
    for (int i = m+1; i <= r; i++) if (ponta[i]+1 and ponta[i] < l)
        data::add(qu[i]), qnt++;
    solve(l, m);
    while (--qnt) data::rollback();
    for (int i = l; i <= m; i++) if (ponta[i]+1 and ponta[i] > r)
        data::add(qu[i]), qnt++;
    solve(m+1, r);
    while (qnt--) data::rollback();
}
```